

单位代码： 10293 密 级： _____

南京邮电大学

专业学位硕士学位论文



论文题目： 基于程序切片和深度学习的代码漏洞检测

学 号 1220045111

姓 名 卢跃

导 师 张迎周

专业学位类别 电子信息硕士

类 型 全日制

专业（领域） 软件工程

论文提交日期 二〇二三年六月

Vulnerability Detection Based on Program Slicing and Deep Learning

Thesis Submitted to Nanjing University of Posts and
Telecommunications for the Degree of
Master of Engineering



By

Yue Lu

Supervisor: Prof. Yingzhou Zhang

June 2023

摘要

随着互联网的普及和软件的广泛应用，漏洞的存在可能导致严重的安全威胁和损失。在这个背景下，漏洞检测技术的发展和创新的成为了保障软件系统安全性的关键。软件开发的快速发展，软件迭代速度越来越快，这意味着漏洞检测技术需要能够快速适应新的软件版本和更新。深度学习模型可以通过学习大量的代码和漏洞样本，从中提取特征和模式，并准确地识别和预测潜在的漏洞。然而，现有的基于深度学习的漏洞检测方法检测的粒度较粗，通常是文件级、函数级或行级别，存在准确率较低、误报率较高等问题。为了克服这些挑战并提升漏洞检测技术的性能，本研究关注于深度学习和程序切片结合的方法。

针对现有基于深度学习和程序切片的漏洞检测方法仅获取代码的语义信息，存在可能无法全面准确地捕捉到潜在的漏洞的问题，研究了一种基于 LLVM IR 的静态切片语义与度量代码的漏洞检测方法。提出了基于 LLVM IR 的漏洞兴趣点检测算法，使用符号化切片工具 SymPas 来获取兴趣点的切片，通过指令级嵌入方法 Inst2vec 来获取 IR 指令的向量表示，并融合指令级切片度量紧密度、重叠度、覆盖度和认知复杂度度量，可以深入分析切片语句之间的关系和特征。构建了混合模型 ResCNN-GRU 来进行训练，将提取的特征进行有效融合和学习。这种混合模型能够充分利用切片的语义信息和度量信息，进一步提高了漏洞检测的性能。本文方法使用 LLVM IR 作为中间表示语言具有很好的适应性，充分利用切片语义和度量信息，在准确率和漏报率上有较好的表现。

针对上述静态切片方法在漏洞检测中存在一些局限性，尤其是在发现特定执行条件下的漏洞方面存在不足之处，研究了基于动态切片与预训练模型的代码漏洞检测方法。动态切片技术能够反映程序的实际执行情况，它能够发现一些只有在特定执行条件下才会出现的漏洞。此外，动态切片方法能够将程序表示为更小粒度的代码块，从而更详细地捕捉代码的语义特征。使用 CodeBERT 预训练模型将程序切片语句块表示为二维张量。把这些切片块视为灰度图像，将代码的结构和语义信息编码为像素值，能够更全面地捕捉代码的语义特征，包括局部结构和上下文信息。此外，借助计算机视觉模型 Swin Transformer 的特征提取能力，进一步从图像中提取关键的漏洞特征。这种方法可以更准确地进行漏洞检测，因为计算机视觉模型能够有效地学习图像中的重要特征，从而帮助准确地识别代码中的缺陷。通过结合动态切片技术和预训练模型的优势，本文的方法能够更全面地捕捉代码的语义特征和关键漏洞特征，提高代码漏洞检测的准确性并降低了误报率。

本文通过实验验证了本研究提出的两种方法，实验结果表明本文的方法取得了较好的效果。降低了误报率和漏报率，提高了漏洞检测的准确性和可靠性，而且具有较好的适应性，

拓展了漏洞检测的应用范围。

关键词：静态切片，切片度量，预训练模型，动态切片, Swin Transformer

Abstract

With the widespread use of the Internet and the extensive application of software, the presence of vulnerabilities can lead to serious security threats and losses. In this context, the development and innovation of vulnerability detection techniques have become crucial for ensuring the security of software systems. The rapid development of software and the increasing speed of software iteration imply that vulnerability detection techniques need to quickly adapt to new software versions and updates. Deep learning models can extract features and patterns by learning from a large amount of code and vulnerability samples, accurately identifying and predicting potential vulnerabilities. However, existing deep learning-based vulnerability detection methods operate at a coarse granularity, typically at the file, function, or line level, which results in lower accuracy and higher false positive rates. To overcome these challenges and improve the performance of vulnerability detection techniques, this study focuses on the combination of deep learning and program slicing.

In response to the problem that existing code vulnerability detection methods based on program slicing only capture the semantic information of the code, which may lead to incomplete and inaccurate identification of potential vulnerabilities, a static slicing semantic and metric-based vulnerability detection method using LLVM IR is studied. An algorithm for detecting vulnerability points based on LLVM IR is proposed. The symbolic slicing tool SymPas is employed to obtain slices of interest, and the Inst2vec method is used to obtain vector representations of LLVM IR instructions. The method combines instruction-level slicing metrics such as density, overlap, coverage, and cognitive complexity to analyze the relationships and features between slicing statements in depth. A hybrid model, ResCNN-GRU, is constructed for training, effectively integrating and learning the extracted features. This hybrid model can make full use of the semantic and metric information of the slices, thereby further improving the performance of vulnerability detection. The proposed approach utilizes LLVM IR as an intermediate representation language, demonstrating good adaptability and achieving favorable results in terms of accuracy and false negative rate by leveraging the semantic and metric information of the slices.

In response to the limitations of current static slicing methods in code vulnerability detection, especially in discovering vulnerabilities under specific execution conditions, a code vulnerability detection method based on dynamic slicing and pre-trained models is studied. Dynamic slicing technology can reflect the actual execution behavior of a program, making it more realistic and

accurate. It can discover vulnerabilities that only occur under specific execution conditions. Furthermore, dynamic slicing methods can represent programs as smaller granularity code blocks, capturing the semantic features of the code in more detail. The CodeBERT pre-trained model is used to represent the sliced code blocks as two-dimensional tensors. These sliced blocks are treated as grayscale images, encoding the structure and semantic information of the code as pixel values. The benefit of this approach is the ability to capture the semantic features of the code more comprehensively, including local structure and contextual information. Additionally, leveraging the feature extraction capabilities of the computer vision model Swin Transformer, critical vulnerability features are further extracted from the images. This method enables more accurate vulnerability detection since the computer vision model can effectively learn important features in the images, helping identify defects in the code accurately. By combining the advantages of dynamic slicing technology and pre-trained models, the proposed method comprehensively captures the semantic features of the code and key vulnerability features, thereby improving the accuracy of code vulnerability detection and reducing false positive rates.

The two proposed methods in this study were experimentally validated, and the results demonstrate their effectiveness in code defect detection. The experiments showed a significant reduction in false positives and false negatives, thereby improving the accuracy and reliability of vulnerability detection. Moreover, these methods exhibited good adaptability, expanding the application scope of vulnerability detection.

Key words:Static Slicing, Slice Metrics, Pre-trained Model, Dynamic Slicing, Swin Transformer

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.2.1 传统的漏洞检测.....	2
1.2.2 基于机器学习的漏洞检测.....	4
1.3 研究内容及创新点.....	6
1.4 论文组织结构.....	8
第二章 相关背景知识介绍	10
2.1 卷积神经网络.....	10
2.2 Transformer 模型.....	10
2.3 文本表示技术.....	11
2.3.1 One-hot 编码.....	11
2.3.2 词袋模型.....	12
2.3.3 TF-IDF	12
2.3.4 词嵌入模型.....	12
2.4 程序切片相关知识.....	13
2.5 数据流分析.....	14
2.6 LLVM IR.....	16
2.7 本章小结.....	17
第三章 基于静态切片语义与度量的代码漏洞检测	18
3.1 相关工具和总体框架.....	18
3.1.1 程序切片工具.....	18
3.1.2 总体框架.....	20
3.2 切片兴趣点.....	21
3.2.1 缓冲区溢出相关漏洞.....	21
3.2.2 空指针解引用漏洞.....	23
3.2.3 SQL 注入漏洞	25
3.2.4 函数调用漏洞.....	27
3.2.5 基于 LLVM IR 的漏洞兴趣点检测算法	28
3.3 提取语义信息.....	30
3.4 提取切片度量特征.....	33
3.5 深度学习模型构建.....	35
3.6 本章小结.....	39
第四章 基于动态切片与预训练模型的代码漏洞检测	40

4.1 总体框架.....	40
4.2 提取动态切片.....	41
4.2.1 动态切片算法.....	41
4.2.2 兴趣点的动态切片生成.....	43
4.3 代码嵌入.....	45
4.3.1 CodeBERT 预训练模型	45
4.3.2 基于 CodeBERT 和程序切片的代码表示算法	49
4.4 基于 Swin Transformer 模型的漏洞分类	50
4.5 本章小结.....	53
第五章 实验结果与分析	54
5.1 基于静态切片语义与度量代码的漏洞检测的实验与分析	54
5.1.1 实验数据集选取.....	54
5.1.2 评估指标.....	54
5.1.3 实验环境及参数设置.....	55
5.1.4 实验结果分析.....	56
5.2 基于动态切片与预训练模型的代码漏洞检测的实验与分析	58
5.2.1 实验数据集选取.....	58
5.2.2 实验环境.....	59
5.2.3 实验结果分析.....	59
5.3 本章小结.....	65
第六章 总结与展望	66
6.1 工作总结.....	66
6.2 研究展望.....	67
参考文献.....	68

第一章 绪论

1.1 研究背景及意义

在当今互联网时代，软件应用程序已经成为人们日常工作和生活中不可或缺的一部分。然而，由于软件开发人员的疏忽、错误的设计、实现和测试，导致软件应用程序中常常存在各种安全漏洞，漏洞是指软件、硬件或系统中的错误，可以被利用来破坏系统的安全性。漏洞一旦被攻击者利用，就可能导致机密信息泄露、拒绝服务攻击、远程代码执行等安全问题。因此，漏洞检测成为了软件开发、系统维护、网络安全等领域中非常重要的一项技术。近些年来，各种规模的软件漏洞事件频繁发生，给企业、政府和个人带来了严重的损失和安全风险。例如，2017 年“永恒之蓝”漏洞，该漏洞利用微软操作系统中的一个漏洞，通过网络攻击来传播。在全球范围内，该漏洞被利用来攻击大约 20 万台计算机，并造成了大量的数据损失和服务中断；2021 年 3 月，微软 Exchange Server 出现了四个漏洞，被称为“ProxyLogon”漏洞，攻击者可以通过这些漏洞远程访问 Exchange Server，窃取敏感信息或者加密文件要求赎金，影响了全球数百万 Exchange Server 用户；在 2022 年 3 月，Spring 框架被披露出一项名为 CVE-2022-22965 的远程代码执行（RCE）0day 漏洞。安全研究人员发现，一旦攻击者成功利用该漏洞，便能够远程执行代码，进而对目标主机进行后门文件的写入和配置修改。通过操纵后门文件，攻击者可以获得目标主机的权限，并对整个系统进行攻击。这些事件再次提醒我们，软件漏洞检测和安全评估是企业、机构和个人必须关注和重视的事情。在这个数字化和互联网时代，漏洞检测技术的发展对于保护信息安全、维护社会稳定和发展具有重要的意义。因此，对于软件安全的保护和漏洞检测变得尤为重要。漏洞检测的主要任务是识别软件程序中潜在的安全漏洞，从而及早发现和修复这些漏洞，防止它们被利用。漏洞检测不仅能够提高软件的安全性，减少黑客攻击的风险，而且也能够降低软件维护成本，提高软件的质量和可靠性。

传统的漏洞检测方法包括静态分析、动态测试和人工审查等，静态分析是指在不运行程序的情况下对程序进行分析，以发现其中潜在的漏洞。静态分析的方法包括符号执行、抽象解释、模型检测、程序切片等^[1]。其中符号执行方法对程序的执行路径进行完整的覆盖，但是其缺点是路径爆炸问题，导致对大规模程序分析的困难^[2]；抽象解释方法则是对程序的抽象语义进行分析，但是其精度和可扩展性都有一定的限制。动态测试是指在运行程序时通过输入和输出的监控来发现漏洞，包括黑盒测试和白盒测试。其中黑盒测试只关注程序的输出结果，

而忽略了内部执行过程中的状态变化，因此其检测精度不高；白盒测试则需要获取程序内部状态，因此需要侵入式的测试，也存在运行时开销较大的问题。人工审查则是通过人工对程序进行代码审查或者测试来发现漏洞，其优点是可以对程序的某些特定部分进行深入分析，但是效率较低，同时容易出现疏漏。这些传统的漏洞检测方法都存在一些问题，如精度低、效率慢、覆盖范围有限等。随着软件规模和复杂度的增加，这些问题更加突出，需要寻找新的方法来提高漏洞检测的效率和精度。传统的漏洞检测方法通常需要安全专家进行人工定义规则。这些规则通常基于漏洞的特征或者已知的攻击模式，并被用于指导漏洞检测工具的行为。这些规则包括正则表达式、特定的代码模式、已知的漏洞代码等等。这种方法的优点是容易实现和使用，同时也比较精确，可以发现大量的已知漏洞。但是，这种方法也存在一些问题。首先，由于漏洞数量庞大且不断增加，人工定义规则变得越来越困难。其次，这种方法往往只能发现已知的漏洞，而无法发现新的未知漏洞。此外，这种方法的准确性也取决于安全专家定义规则的质量和准确性。最后，由于软件的复杂性不断提高，这种方法往往需要消耗大量的时间和精力，而且很难保证完全覆盖所有漏洞。这些传统的漏洞检测方法都存在一些问题，如精度低、效率慢、覆盖范围有限等。随着软件规模和复杂度的增加，这些问题更加突出，需要寻找新的方法来提高漏洞检测的效率和精度。因此，为了解决这些问题，近年来越来越多的研究关注于基于深度学习的漏洞检测方法，传统的漏洞检测方法需要安全专家进行人工定义规则，这样会产生大量的人工成本，并且漏洞检测的准确性和可靠性受到人工规则的限制。而基于深度学习的漏洞检测方法可以自动学习和识别漏洞的特征，无需手动定义规则，使得漏洞检测过程更加自动化和高效，减少了对专家知识的依赖，提高了检测的准确性和可扩展性。

1.2 国内外研究现状

1.2.1 传统的漏洞检测

传统的漏洞检测可分为静态分析、动态分析和混合分析方法。静态分析是指在不执行程序的情况下对程序进行分析，动态分析是指在程序运行时对程序进行分析，混合方法是指将静态和动态分析结合在一起。

基于规则的漏洞检测是最早也是最常见的漏洞检测方法之一。该方法通过事先定义一些规则或模式，来检测代码中是否存在违反规则或模式的部分，进而判断代码是否存在漏洞。

Flawfinder^[4]是一种基于规则的漏洞检测开源工具，其使用了一个包含了超过 600 个规则的规

则集合, 这些规则可以检测出 C、C++、Java、JavaScript、Perl、PHP、Python 和 Ruby 等语言中的常见漏洞。基于规则的检测方法可以在不执行程序的情况下进行检测, 因此其速度较快。Flawfinder 的缺点是其只能检测出已知的漏洞类型, 且由于其规则集合较为静态, 无法及时更新, 可能无法检测出最新的漏洞类型。Checkmarx^[5]是一款商业产品, 提供源代码分析、数据流分析和控制流分析等功能, 可以检测出多种类型的漏洞, 包括 SQL 注入、跨站脚本攻击、命令注入等。相较于开源工具, Checkmarx 的解析能力更强, 能够进行更为深入的源代码分析, 并且不需要编译, 直接对源代码进行分析, 能够提高分析效率。还有一些其他基于规则的工具如 Cppcheck、RATS、Fortify、Coverity^[6-9]等, 基于规则的漏洞检测的优点是实现简单, 运行速度快, 但是缺点是只能检测已知的漏洞和错误模式。

符号执行 (Symbolic Execution) 是一种高级的静态分析技术, 其基本思想是通过对程序路径的符号表示来检查程序的执行路径, 以发现程序中可能存在的漏洞或错误。Ramos 和 Engler^[10]在 2015 年提出了一种名为“Under-Constrained Symbolic Execution”的符号执行变体, 旨在通过直接检查单个函数而不是整个程序来提高可扩展性, 从而发现软件中的漏洞。他们还提出了 UC-KLEE 框架, 该框架用于检查 C/C++ 系统代码, 并且包含检查程序是否引入崩溃的功能, 还能实现内存泄漏、未初始化数据和不安全用户输入等检查。该方法被证明在 BIND、OpenSSL 和 Linux 内核等软件中可以找到多个漏洞, 并且可以验证代码的正确性。Liang^[11]等人提出了一种基于 LLVM 中间表示的静态分析工具 MLSA, 可以分析多种编程语言的程序, 结合符号执行和约束求解器 (Z3 SMT solver), 用于查找程序中的错误。目前, MLSA 可以检测一些常见的错误类型, 例如除零错误、指针溢出和死代码。此外, 作为一个框架, MLSA 遵循可扩展性和可伸缩性的原则, 可以帮助检测其他类型的错误。实验证明, MLSA 在真实世界的软件中可以有效地发现错误。符号执行技术可以发现代码中的未知漏洞, 但是由于符号执行的路径爆炸问题, 需要解决路径约束求解的问题, 导致运行速度较慢, 并且存在路径爆炸问题, 导致需要处理大量的路径约束, 难以应用于大规模程序的分析。

模糊测试 (Fuzz Testing) 是当前流行的一种动态分析技术, 常用于发现程序中的漏洞。模糊测试通过输入大量的随机、半随机或智能生成的测试数据来对程序进行测试, 以寻找可能存在的错误、异常或崩溃情况。模糊测试技术已经被广泛应用于软件开发、安全测试、漏洞挖掘和自动化测试等领域。Ganesh 等人^[12]提出了一种新的自动化白盒模糊测试技术和实现该技术的工具 BuzzFuzz, 定向模糊测试技术使用污点来自动发现和利用有关输入文件格式的信息。Zhang^[13]等人在 2019 年的研究中提出了一种自动化的测试框架 SRFuzzer, 用于发现物理 SOHO 路由器设备的多种漏洞类型。该框架通过使用两个输入语义模型, 即键值数据模型和 CONF-READ 通信模型, 连续有效地生成测试用例, 并利用电源管理自动恢复测试环境。

它协调多种变异规则和多种监视机制以触发多种漏洞类型。模糊测试是一种黑盒测试方法，可以发现未知漏洞，也就是 0-day 漏洞，而且在漏洞挖掘方面的效果也比较好。但是模糊测试需要大量的计算资源和时间，而且可能会在长时间内未发现有效的漏洞，从而导致测试效率低下，由于测试用例的生成是随机的，可能会导致测试用例的质量不高，测试用例生成的路径不够完整，从而导致代码覆盖率不高。

混合方法指的是将静态和动态分析结合起来，以达到综合利用两种方法的优势的目的。通常，这种方法将静态分析的结果用作动态分析的输入，或者将动态分析的执行路径反馈到静态分析中进行更深入的分析。Amin^[14]等人所提出的 Androshield 方法，该方法是一种自动化 Android 应用程序漏洞检测方法，使用了静态分析和动态分析的混合方法。该方法通过对应用程序的代码进行分析和监视，能够检测出多种安全漏洞，为 Android 应用程序的安全性提供了有效的保障。Padmanabhuni^[15]和 Tan 提出了一种名为“混合静态-动态分析”的方法来审计缓冲区溢出漏洞。他们使用简单规则生成测试数据，然后通过动态分析来确认一些漏洞。对于那些未被动态分析确认的漏洞，他们利用静态代码属性进行预测。

本文的方法基于深度学习模型，能够有效地识别潜在的漏洞。为了提高检测的准确性和可靠性，结合了紧密度、重叠度、覆盖度和认知复杂度等切片度量，这种将手工设计的特征与深度学习自主学习的代码漏洞特征相结合的方式，可以提升漏洞检测的准确性。

1.2.2 基于机器学习的漏洞检测

机器学习在代码漏洞检测方面已经取得了一些进展。通常情况下，基于机器学习的方法将代码漏洞检测视为一个二进制分类问题，即判断给定的代码片段是否存在漏洞。早期的研究工作通常依赖于专家经验来构建手工制作的特征，然后将这些特征输入到机器学习算法中进行漏洞检测。Nguyen 和 Tran^[16]提出了一种基于依赖图的方法来预测易受攻击的软件组件。他们的研究表明，通过分析软件组件之间的依赖关系，可以发现潜在的脆弱性和安全风险。在他们的工作中，他们构建了依赖图并使用一种预测模型来识别易受攻击的组件。Shin^[17]等人在其研究中评估了复杂性指标、代码变动指标和开发人员活动指标作为软件漏洞的指示性指标。他们的研究目的是探索这些指标是否可以用于预测软件系统中的漏洞存在。Perl^[18]等人提出了一种名为 Vccfinder 的方法，通过结合代码指标分析和从代码库收集的元数据来帮助代码审查团队优先处理其工作。该方法的误报率比其他类似系统低，并通过大规模映射 CVE 与 GitHub 提交之间的关系创建了易受攻击提交的数据库。他们训练了一个支持向量机 SVM 分类器来标记可疑的提交，该方法在相同的召回率下减少了超过 99% 的误报率。传统机器学习

漏洞检测方法通常需要依靠专家经验手工设计和提取特征。这种方法需要深入了解代码漏洞的特性，并手动选择和构建相应的特征集。然而，代码漏洞的特性往往复杂多样，难以通过手工方式完全捕捉到所有的漏洞模式，因此手工特征工程可能无法充分表达漏洞。

为了解决传统机器学习漏洞检测的问题，研究人员探索了使用机器学习技术从代码中自动学习特征的方法。这些方法利用深度学习、自然语言处理等技术，从代码片段、抽象语法树或控制流图中提取有意义的表示。通过捕捉代码内部的模式和关系，机器学习模型能够有效地识别潜在的漏洞，而无需依赖手工制作的特征。Hin D 等人^[19]提出了一种新的深度学习框架 LineVD，它将语句级漏洞检测看作是节点分类任务。利用图神经网络来捕捉函数中语句之间的控制和数据依赖关系，并使用基于转换器的模型对原始源代码令牌进行编码。尤其重要的是，通过解决函数级和语句级信息之间的输出冲突，LineVD 显著提高了对函数代码的预测性能。Yamaguchi 等人^[20]提出了一种使用抽象语法树进行广义漏洞推广的方法。该方法旨在辅助安全分析人员在源代码审计过程中发现漏洞。论文中的方法通过提取代码的抽象语法树并确定其中的结构模式，将每个函数描述为这些模式的组合。这种表示方法使得已知的漏洞可以被分解，并推广到代码库中，从而可以向分析人员提供可能受到相同缺陷影响的函数。Li 等^[21]首次提出了使用基于深度学习的漏洞检测方法，以减轻人工专家手动定义特征的繁琐和主观性任务。使用代码片段（Code Gadgets）来表示程序，并将其转化为向量表示，设计和实现了一个名为 VulDeePecker 的深度学习漏洞检测系统，相比其他方法，VulDeePecker 具有更低的漏报率。Li 等人^[22]提出了一种名为 SySeVR 的框架，用于使用深度学习来检测软件漏洞。该框架专注于获取能够适应与漏洞相关的语法和语义信息的程序表示，可以检测数组、算术表达式、指针、函数调用相关的四类漏洞类型。Yamaguchi 等人^[23]的方法引入了一种新的源代码表示形式，即代码属性图，它将抽象语法树、控制流图和程序依赖图等经典程序分析概念合并到一个联合数据结构中。这种综合表示形式使得可以通过图形遍历的方式来优雅地建模常见漏洞模板，如缓冲区溢出、整数溢出、格式化字符串漏洞和内存泄漏等。Zhou 等人^[24]提出了一种名为 Devign 的方法，它是基于通用图神经网络的图级分类模型，用于学习代码属性图中丰富的代码语义表示。该方法通过一个新颖的卷积模块从学习到的丰富节点表示中提取有用特征，并用于图级分类。Cao 等人^[25]提出了一种名为 MVD 的方法，它基于流敏感图神经网络（FS-GNN），用于检测与内存相关的漏洞。FS-GNN 能够联合嵌入非结构化信息和结构化信息，以捕获与隐式内存相关的漏洞模式。Wu 等人^[26]实现了一个名为 VulCNN 的模型，借鉴了基于深度学习的图像分类方法，将函数的源代码转换为图像表示，将源代码解析和标记化，并保留了关键的句法和语义信息，以便模型能够有效地学习源代码的内部表示。Rabheru 等人^[27]设计了 DeepTective 用于检测 PHP 源代码中的漏洞，采用了一种新颖的混合

技术,结合了门控循环单元和图卷积网络,以利用句法和语义信息来检测 SQLi、XSS 和 OSCI 漏洞。DeepTective 为了学习到源代码中学习句法和结构特征,将其转换为一系列要由门控循环单元 (GRU) 进行分析的标记。GRU 是一种与 LSTM 相关的神经网络,能够嵌入有关代码结构的顺序信息。创新之处在于,通过使用图卷积网络 (GCN) 分析控制流图 (CFG) 进而学习源代码的语义属性。序列模型和图形模型在代码漏洞检测中表现相当,但它们也有各自的优势和限制。序列模型更适合处理局部上下文和语法信息,而图形模型能够更好地处理全局依赖和结构信息。但图结构在表示源代码的同时,也引入了复杂的计算和分析过程。图的构建和处理通常需要大量的计算资源和时间。这可能导致在大规模代码库或复杂代码中的漏洞检测过程中的性能下降。基于图结构的漏洞检测方法通常需要额外的存储空间来表示代码的图形表示形式。对于大型代码库,这可能需要大量的内存和存储空间。不同的编程语言和代码库可能具有不同的语法结构和代码规范,这可能导致在应用基于图结构的漏洞检测方法时需要针对特定语言或代码库进行调整和优化。这限制了这些方法的通用性和可扩展性。尽管图结构可以表示代码的结构和依赖关系,但在某些情况下,它们可能无法完全捕捉到上下文信息。某些漏洞可能依赖于代码执行的特定路径或条件,这些上下文信息可能不容易在图结构中表示。

现有深度学习方法在漏洞检测领域的应用中存在一些局限性。其中,模型的输入粒度通常是以文件级、函数级或行级,这种粒度相对较粗,可能无法捕捉到漏洞的特征。此外,这些方法往往没有考虑到某些漏洞可能依赖于代码执行的特定路径,导致漏洞的检测结果不够准确。为了解决这些问题,本文利用动态切片能够反映程序的实际执行情况,更真实和准确地发现一些只有在特定执行条件下才会出现的缺陷。此外,动态切片方法将程序表示为更小粒度,使得分析更加细致。结合预训练模型 CodeBERT,预训练模型通过大规模的源代码数据进行自监督学习,从而学习到代码的语义表示。这使得在理解代码的语义和结构方面具有强大的能力,能够捕捉到代码中的上下文信息和语法结构。

1.3 研究内容及创新点

本文分别针对静态切片和动态切片在代码漏洞检测领域的应用做了相关研究,主要研究内容如下:

(1) 针对现有基于程序切片的漏洞检测方法仅获取代码的语义信息,存在可能无法全面准确地捕捉到潜在的漏洞的问题,研究了一种基于 LLVM IR 的静态切片语义与度量代码的漏洞检测方法。提出了基于 LLVM IR 的漏洞兴趣点检测算法,使用符号化切片工具 SymPas 来

获取兴趣点的切片,通过指令级嵌入方法 Inst2vec 来获取 IR 指令的向量表示,并融合指令级切片度量紧密度、重叠度、覆盖度和认知复杂度度量,可以深入分析切片语句之间的关系和特征。构建了混合模型 ResCNN-GRU 来进行训练,将提取的特征进行有效融合和学习。这种混合模型能够充分利用切片的语义信息和度量信息,进一步提高了漏洞检测的性能。本文方法使用 LLVM IR 作为中间表示语言具有很好的适应性,充分利用切片语义和度量信息,在准确率和漏报率上有较好的表现。

(2) 针对当前静态切片方法在漏洞检测中存在一些局限性,尤其是在发现特定执行条件下的漏洞方面存在不足之处。研究了基于动态切片与预训练模型的代码漏洞检测方法。动态切片技术能够反映程序的实际情况,更加真实和准确。它能够发现一些只有在特定执行条件下才会出现的漏洞。此外,动态切片方法能够将程序表示为更小粒度的代码块,从而更详细地捕捉代码的语义特征。使用 CodeBERT 预训练模型将程序切片语句块表示为二维张量。把这些切片块视为灰度图像,将代码的结构和语义信息编码为像素值。这样做的好处是能够更全面地捕捉代码的语义特征,包括局部结构和上下文信息。此外,借助计算机视觉模型 Swin Transformer 的特征提取能力,进一步从图像中提取关键的漏洞特征。这种方法可以更准确地进行漏洞检测,因为计算机视觉模型能够有效地学习图像中的重要特征,从而帮助准确地识别代码中的缺陷。通过结合动态切片技术和预训练模型的优势,本文的方法能够更全面地捕捉代码的语义特征和关键漏洞特征,提高代码漏洞检测的准确性并降低了误报率。

本文的创新点如下:

(1) 本研究提出的基于 LLVM IR 的漏洞兴趣点检测算法。针对数组变量使用、指针变量使用、算术表达式以及 API/库函数调用等四种类型的指令,提出了兴趣点选取的准则,这些准则提供了指令中兴趣点选取的依据。将目标程序转化为 LLVM IR 指令的中间代码表示,利用这些指令创建一个函数调用图,用于记录函数之间的调用关系。通过遍历函数调用图中的每个函数、函数内的基本块以及基本块内的每条 LLVM IR 指令,对指令进行六大兴趣点准则的判断。若指令满足兴趣点准则,最终从切片表中获取与该指令相关的兴趣点。相较于现有方法基于抽象语法树来选取兴趣点,基于 LLVM IR 的中间表示具有平台无关性、可扩展性和中间表示优化等优势,能够更准确、全面地发现程序中的兴趣点。

(2) 提出了一种在 LLVM IR 层面的静态切片语义与度量代码的漏洞检测方法。该方法利用符号化切片工具 SymPas 获取创新点 1 中兴趣点的 IR 切片,并使用 Inst2vec 获取 IR 指令的向量表示。同时,融合了指令级切片度量,包括紧密度、重叠度、覆盖度和认知复杂度。本研究引入了认知复杂度作为一种新型的度量方式,用于衡量了解某个切片的目的数据流信息和控制流信息所需的认知努力,从而判断该切片存在缺陷的可能性。通过引入指令级切片

度量，能够提高漏洞检测的准确率。为了有效地融合和学习提取的特征，本方法采用了混合模型 ResCNN-GRU 进行训练，进一步提高了检测的准确性和可靠性。相较于现有方法仅获取切片的语义信息，本文的漏洞检测方法在 LLVM IR 层面能够更准确地捕捉程序中的缺陷。通过获取兴趣点的切片语义信息并融合指令级切片度量，提高了检测的精确率和可信度。同时，利用混合模型进行训练进一步改善了检测性能。

(3) 提出了基于动态切片与预训练模型的代码漏洞检测方法。相比于现有的基于静态切片的代码漏洞检测方法，本文利用动态切片能够反映程序的实际情况，更真实和准确地发现一些只有在特定执行条件下才会出现的缺陷。此外，动态切片方法将程序表示为更小粒度，使得分析更加细致。本文使用了 CodeBERT 预训练模型将程序切片语句块表示为二维张量，提出了基于 CodeBERT 和程序切片的代码表示算法，通过这种灵活的方式调整代码表示的长度，算法能够适应不同长度的程序切片，并确保关键信息被充分保留，为后续的模型训练和漏洞检测提供了方便和可靠的输入，然后将这个张量视为灰度图像，并将代码的结构和语义信息编码为像素值，从而更全面地捕捉代码的语义特征。借助 Swin Transformer 的特征提取能力，能够进一步提取图像中的关键漏洞特征，从而更准确地进行漏洞检测。

1.4 论文组织结构

本文的组织架构如图 1.1 所示：

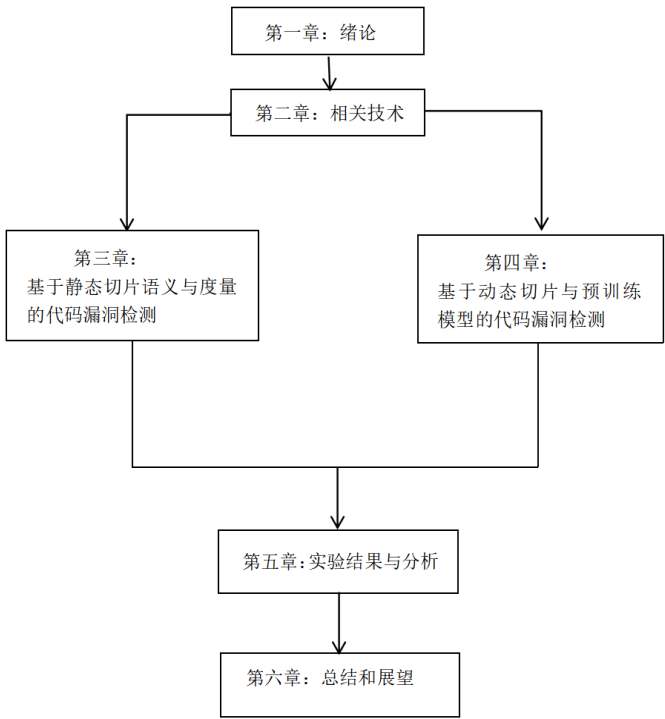


图 1.1 论文组织架构图

文章一共分为六章，具体安排如下所示：

第一章：绪论。首先进行了文章研究背景和研究意义的阐述，结合近年软件漏洞频繁发生的事件强调了漏洞检测的重要性，然后分别从传统漏洞检测方法和基于机器学习的漏洞检测方法叙述了漏洞检测技术在国内外的研究现状，最后简短地说明了文章的创新点和组织结构。

第二章：相关背景知识介绍。介绍了本文用到的相关技术，卷积神经网络、Transformer 模型、文本表示技术、程序切片、数据流分析、LLVM 框架。

第三章：基于静态切片语义与度量代码的漏洞检测，本章首先举例缓冲区溢出漏洞、空指针解引用漏洞、SQL 注入攻击漏洞、函数调用漏洞四种类型在 LLVM IR 层面兴趣点选取规则，并提出了基于 LLVM IR 缺陷的兴趣点检测算法，通过 SymPas 可以获取兴趣点的前向切片和后向切片，用 Inst2vec 获取 IR 指令的向量表示，并融合指令级切片度量，最终通过混合模型 ResCNN-GRU 进行训练。

第四章：基于动态切片与预训练模型代码的漏洞检测方法，首先，利用简化动态依赖图（RDDG）方法获取更精确的代码切片。然后使用 CodeBERT 对这些切片语句进行编码，并提出了一种基于 CodeBERT 和程序切片的代码表示算法。接下来，将这些代码表示视为单通道灰度图像，并通过 Swin Transformer 进行训练，以提取漏洞特征。

第五章：本章对第三章的基于静态切片语义与度量代码的漏洞检测方法和第四章的基于动态切片与预训练模型的代码缺陷检测方法分别进行了实验，并对实验结果进行了分析。

第六章：总结和展望。总结了本文提出的方法，包括其创新点和不足，且针对其不足对未来研究内容和工作做了进一步的展望。

第二章 相关背景知识介绍

本章首先介绍了深度学习相关的模型，如卷积神经网络，Transformer 模型，然后介绍了文本表示技术，分别从 One-hot 编码、词袋模型、TF-IDF、词嵌入模型四个方面详细介绍，然后介绍了程序切片、数据流分析、LLVM IR 相关背景知识，为本文论文工作做好理论铺垫。

2.1 卷积神经网络

卷积神经网络（Convolutional Neural Network）是一种广泛应用于图像、视频、语音、文本等领域的神经网络模型^[29]。与传统的神经网络不同，卷积神经网络利用卷积操作对输入数据进行特征提取和处理，从而实现对输入数据的分类和识别。它的基本结构包括卷积层、池化层、全连接层等。其中，卷积层主要用于提取输入数据的局部特征，池化层用于降低特征图的维度，全连接层则将提取到的特征进行分类和识别。卷积神经网络的输入一般为一张图片，通过多个卷积层和池化层的处理，最终得到一组特征向量，再通过全连接层进行分类或回归。卷积神经网络的训练通常采用反向传播算法进行参数优化和更新，从而使网络能够更好地拟合训练数据和进行分类和识别。在训练过程中，需要选择适当的损失函数、学习率、优化器等参数，并对网络结构进行调整，以提高模型的性能和泛化能力。卷积神经网络在计算机视觉、语音识别、自然语言处理等领域有广泛应用。在图像分类中，卷积神经网络可以通过卷积和池化层提取图像的特征，并通过全连接层将特征映射到不同的类别。在目标检测中，卷积神经网络可以将图像分成多个区域，并对每个区域进行分类和回归，从而实现对目标的检测和定位。在自然语言处理中，卷积神经网络可以将文本转换为向量表示，并通过卷积层提取局部特征，再通过全连接层进行分类或回归。

2.2 Transformer 模型

在自然语言处理领域中，传统的序列到序列模型存在着梯度消失和梯度爆炸等问题，限制了模型的深度和性能。为了解决这些问题，Google 在 2017 年提出了 Transformer 模型^[30]，它基于注意力机制来实现序列到序列的建模，不需要使用循环结构，大大提高了模型的并行度和速度，同时也提升了模型的表现。因此，Transformer 模型也成为了自然语言处理领域的一个重要突破。Transformer 模型由编码器和解码器组成，每个部分都有多个 Transformer 层。每个 Transformer 层由多头自注意力机制和前馈神经网络组成。编码器用于将输入序列编码为

一系列上下文相关的表示，解码器用于根据编码器的输出和之前的输出生成下一个输出。其中，自注意力机制可以在输入序列中找到相关的位置，从而将注意力更集中地放在这些位置上。

多头自注意力机制是 Transformer 模型的核心部分，它使模型能够在输入序列中找到相关的位置，并将注意力更集中地放在这些位置上。它由三个矩阵变换得到查询、键和值，这三个矩阵可以使用不同的权重进行变换。通过计算查询和键的点积得到注意力分数，再将注意力分数与值相乘并求和得到自注意力表示。Transformer 模型没有像循环神经网络和卷积神经网络那样的显式顺序信息，因此需要添加一些位置编码来表示单词在序列中的位置。位置编码是一个固定大小的向量，其每个维度都与序列中的一个位置相对应，用于表征该位置的信息。Transformer 模型使用残差连接和层归一化来加速训练和提高模型的表现。残差连接将每个层的输入直接与其输出相加，从而使梯度能够更好地在网络中传播。归一化将每个层的输出归一化，从而使网络更容易训练。

2.3 文本表示技术

文本表示成向量是将自然语言处理中的文本数据转换成向量表示的过程。这是因为计算机无法直接处理文本，需要将其转换成计算机能够处理的数字向量形式。下面介绍几种文本表示成向量技术。

2.3.1 One-hot 编码

One-hot 编码是一种文本表示成向量的技术^[31]，也是一种常见的特征工程方法。在这种方法中，将每个词表示为一个向量，向量的长度为词表中词的总数，向量的值为 0 或 1，其中只有一个位置是 1，其他位置都是 0。这个位置对应词在词表中的索引。例如，假设有一个文本数据集，词表中包含 4 个词：[apple, banana, orange, peach]。那么这四个词的 One-hot 编码：apple: [1, 0, 0, 0]; banana: [0, 1, 0, 0]; orange: [0, 0, 1, 0]; peach: [0, 0, 0, 1]。使用 One-hot 编码的主要优点是它是一种简单而有效的编码方式，不需要考虑词之间的语义关系，仅仅是将每个词映射到一个唯一的向量。此外，One-hot 编码可以在许多机器学习算法中直接使用，如逻辑回归、支持向量机等。然而，One-hot 编码也有其缺点。首先，由于每个向量的长度等于词表大小，当词表很大时，向量的维度会非常高，会导致计算量和存储量的增加。其次，

One-hot 编码不能捕捉到词与词之间的语义关系，因此不能反映词之间的相似性。

2.3.2 词袋模型

词袋模型是自然语言处理中一种常用的文本表示方法^[32]，它将一段文本表示成一个固定长度的向量，用于文本分类、情感分析、信息检索等任务。该模型假设一个文本可以看做一个词的集合，而不考虑词与词之间的顺序关系。词袋模型的步骤如下：首先构建词表：将所有文本中出现过的单词提取出来，并构成一个词表。词表中每个单词都对应着一个唯一的整数编号，作为该单词在向量中的索引。特征提取：对于一段文本，遍历其中的每个单词，统计每个单词在该文本中出现的次数，并记录在对应的向量索引位置上。向量化：将特征提取得到的结果组成一个向量，其中每个维度表示对应单词在文本中出现的次数。这样，每个文本就被表示成了一个固定长度的向量。但是词袋模型也存在一些缺点，丢失了单词的顺序信息，词袋模型将每个单词视为独立的，因此它无法捕捉到单词在句子中的顺序信息。忽略了单词含义的复杂性，词袋模型无法捕捉到单词的含义和语义，因此在某些自然语言处理任务中，如语言生成和机器翻译等，词袋模型的效果并不理想。对高频词和低频词的处理不当，词袋模型对于高频词和低频词的处理都存在一定的问題。

2.3.3 TF-IDF

TF-IDF 是一种基于词袋模型的文本表示成向量方法^[33]。它考虑了单词在整个语料库中的重要性，以及单词在文本中的出现次数。首先，TF（Term Frequency）指的是某个词在文档中出现的频率。计算方式可以是简单计数，也可以使用词频等归一化方式。然后，IDF（Inverse Document Frequency）指的是逆文档频率。如果一个词在大多数文档中都出现过，那么这个词的重要性就会降低。因此，IDF 是对于一个词在文档集中出现的文档频率的倒数。TF-IDF 的计算方式是将一个词在文档中的出现频率和它在整个语料库中的逆文档频率相乘，从而得到该词在文档中的重要性。TF-IDF 的优点是能够识别出在文档中频繁出现但是对于整个文档集合不重要的词语，从而更好地进行文本分类和信息检索。但是，它忽略了词语的顺序和上下文信息。

2.3.4 词嵌入模型

词嵌入（Word Embedding）模型是一种将文本数据转换为低维度向量表示的方法^[34]。它

通过学习每个单词的分布式表示,使得在向量空间中相似的单词距离更近,不相似的单词距离更远,从而实现对单词语义的建模。常见的词嵌入模型有:Word2Vec 是一种基于神经网络的词嵌入模型,通过学习语料库中每个单词与其上下文单词的关系来生成单词的向量表示。Word2Vec 有两种模型结构:连续词袋模型(Continuous Bag of Words, CBOW)和 Skip-Gram 模型。GloVe(Global Vectors)是一种基于矩阵分解的词嵌入模型,它使用全局统计信息来学习单词向量表示,能够更好地处理全局信息,如整个语料库的词频。词嵌入模型的缺点在于,对于不同的任务和应用场景,需要选择不同的词嵌入模型,以便更好地对语义信息进行建模。同时,词嵌入模型对于一些罕见单词或者是新词可能无法准确地进行表示,需要进行一些特殊的处理方法。

2.4 程序切片相关知识

程序切片是一种在程序中抽取出与特定目标语句或变量相关的程序片段的技术,是程序理解和分析的重要手段之一。程序切片技术可以用于程序调试、测试、优化、维护以及程序分析等领域。程序切片技术的基本思想是通过对程序的静态或动态分析,找到程序中与特定目标语句或变量相关的程序片段。其中,静态切片是在不执行程序的情况下,根据程序的源代码分析得到程序片段,而动态切片则是在程序运行时,根据程序执行过程中的变量赋值和控制流等信息得到程序片段。程序切片是一种重要的程序分析理解方法,用于从源程序中抽取对程序中特定点上的特定变量有影响的语句和控制条件,组成新的程序(称作切片),然后通过分析切片来分析源程序的行为。二十多年来,人们对它进行了广泛而深入的研究,取得了许多研究成果,使得它在软件调试、测试、维护、度量、程序变换、软件逆向工程与再工程等方面得到广泛应用,受到了广大软件研究、开发人员的高度重视^[34]。一个程序切片是由程序中的一些语句和判定表达式所构成的集合体。假设制定感兴趣的程序点(即程序中一条语句) p 和变量集合 V 来作为切片准则 $\langle p, V \rangle$,那么所有影响该程序点 p 处的变量 V 的程序语句构成切片。按照是否在切片中考虑程序的具体输入^[35],可以划分为静态切片和动态切片。

静态程序切片的生成算法大致可以分为两类,基于控制流图的数据流方程算法和基于程序依赖图的图可达性算法。在1984年,Mark Weiser提出了程序切片的概念,并在随后的论文中不断完善程序切片理论。他认为,程序的某一个输出只与程序代码中的部分语句有关,因此删除其余的语句并不影响程序的输出结果。在此基础上,他提出了基于控制流图的程序切片理论^[36]。不过Mark Weiser的这种算法精度较低,因为这种算法在沿着CFG向上或向下

遍历的过程中,可能分析到不可执行的程序路径。因此,K.J.Ottensteinz 等^[37]开始提出图可达性算法,以 PDG 取代数据流方程和 CFG 来表达图的节点与源代码语句之前的对应关系。这种方法首先使用节点代替源代码,并进行依赖性分析,添加语句依赖关系的有向依赖边,然后通过约定的切片准则定位所有节点。这些被定位的节点,对应到源代码的语句,就组成了要得到的程序切片。

Horwitz 等^[38]进一步提出了基于系统依赖图的两阶段图可达性算法(简称为 HBR 算法)。HBR 算法可用来计算后向切片,对于系统依赖图的节点 v ,其后向切片为 $BSlice(v)$ 。该算法包含对系统依赖图的两次遍历,关键点在于两次遍历去掉了不同的边。程序切片就是两次遍历所标记的节点的并集。张迎周提出了一种基于轻量级上下文敏感依赖分析的符号化程序依赖性分析方法 SymPas (Symbolic Program Slicing),以 LLVM IR 为目标语言。在过程内通过数据流迭代计算 LLVM IR 指令,变量间的依赖关系。在过程间分析基于的是函数摘要思想,将每个过程(子程序)的切片以符号的形式保存,在每个调用点处进行相应实例化,这样参数间的数据依赖就像过程参数传递一样传播下去,从而可以根据上下文环境,快速获取函数调用过程中形参和实参之间的依赖关系,并且无需构造易爆炸的 SDG 图或超图^[39]。经在相关基准测试程序集上的对比实验, SymPas 兼顾了使用数据流迭代和依赖图进行分析的优点,在不损失精度的情况下,效率明显优于当前最快的 SDG-IFDS 程序依赖性分析方法,并且在空间占用上也有明显的改进。

动态程序切片是一种重要的程序调试方法,它只包含在给定输入下,程序执行路径上所有对程序某个点上的变量有影响的语句。与静态切片相比,动态切片的精确度比较高,但由于动态切片需要动态追踪程序的执行过程,其执行代价较高。执行记录是控制流图中的一个可执行路径,它由程序执行时所经过的节点组成,记作 EH。执行记录中包含语句的顺序与其实际执行顺序相同。任何程序切片都会基于一个准则,称之为切片标准。静态切片标准一般为 $\langle i, V \rangle$,其中 i 为程序执行点, V 为变量集。最初,动态切片的切片准则为 $\langle INPUT, i, V \rangle$, INPUT 为程序中的某个输入。引入执行记录后,切片准则相应地变为 $\langle ij, EH, V \rangle$,其中 EH 为一执行记录, ij 表示 EH 中第 j 个位置上的编号为 i 的语句。动态切片可以根据具体的输入条件和执行路径,动态地计算出程序的切片,因此可以得到更加精确的结果。

2.5 数据流分析

程序切片算法中通常使用数据流分析技术来确定程序执行的依赖关系,从而确定程序切片。数据流分析是一种程序静态分析技术^[40],它通过对程序代码的语法和语义进行建模和分

析，来推导程序中变量和变量之间的依赖关系，从而检测程序中的潜在缺陷和安全漏洞。数据流分析通常会将程序代码分解成一个数据流图，该图表示程序中变量和常量之间的信息流动，然后通过分析数据流图中的信息流，来识别程序中的安全漏洞和缺陷。数据流分析被广泛应用于各种领域，如软件工程、网络安全、数据挖掘等。在软件工程领域，数据流分析可以用于代码重构、代码优化等方面。在网络安全领域，数据流分析可以用于识别程序中的安全漏洞和漏洞攻击，如代码注入、XSS、SQL 注入等。在数据挖掘领域，数据流分析可以用于识别数据中的模式和关系，提取出有价值的信息。

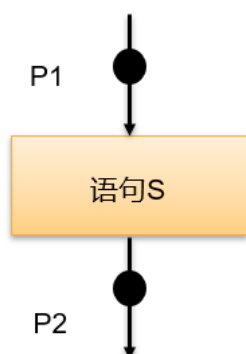


图 2.1 对语句 S 进行前向数据流分析的例子

数据流方程是数据流分析的核心，数据流方程通常是一个由变量和常量组成的方程组，用于描述程序中每个程序点处量和常量的值如何由其他变量和常量计算而来。数据流方程的推导通常基于程序的语法和语义信息，可以通过解方程组来获得程序中每个变量和常量的值。常见的数据流分析算法分为前向数据流分析和后向数据流分析，如图 2.4 所示，我们以前向数据流分析为例对语句 S 进行前向数据流分析，我们需要通过数据流方程获得语句 S 在 P2 程序点处的数据流值。在数据流方程中，有以下几个定义：

- $IN[S]$: 表示流入语句 S 的数据流值；
- $OUT[S]$: 表示流出语句 S 的数据流值，即经语句 S 处的数据流方程计算之后得到的更新之后的语句 S 处的数据流值；
- $GEN[S]$: 表示语句 S 处的数据流值，比如语句 S 为： $x = 5$ ，那么语句 S 处的 $GEN[S]$ 即为 $\{x:5\}$ ，这里用 $:$ 表示程序中变量和值的对应关系；
- $KILL[S]$: 表示在语句 S 处被杀死的变量的集合，即哪些变量的值在该语句执行后会被创建或者更新。比如语句 S 为： $x=5$ ， $IN[S]=\{x:3,y:1\}$ ，那么 $KILL[S]=\{x:3\}$

在图 2.1 中，假设在 P1 处有 $IN[S]$ ，在语句 S 处有 $GEN[S]$ ，则经语句 S 处的数据流方程计算可得 P2 处的数据流值即 $OUT[S] = GEN[S] \cup (IN[S] - KILL[S])$ 。

2.6 LLVM IR

LLVM 是 2000 年起源于 Illinois 大学的一个研究项目，随后进入开源世界。LLVM 的命名最早来源于底层语言虚拟机（Low Level Virtual Machine, LLVM），后来演变为一个建立编译器的基础框架，对程序的整个生命周期，包括编译、链接、执行过程进行优化^[41]。它的优势在于能够提供了大量用于建立编译器的可复用模块，帮助减少编写新编译器的工作量。

LLVM 的整体结构是一个三阶段设计架构，如图 2.2 所示，其主要组件是前端、优化器和后端。前端分析源程序，构建抽象语法树。优化器负责执行各种转换以尝试缩短代码的运行时间。编译器后端的常见部分包括指令选择、寄存器分配和指令调度。

根据三阶段设计的定义，其最大的好处就是如果编译器在优化器中使用某种通用的代码表示形式，那么只需要为想要编译的任意语言编写前端，并为想要的任意目标环境编写后端即可。现有的优化器和后端是可重用的，要为新的编程语言提供支持，只需要实现新的前端，即编写 $N+M$ （ N 个不同的前端， M 个不同后端）个编译器，而不是从头开始编写 $N*M$ 个编译器。

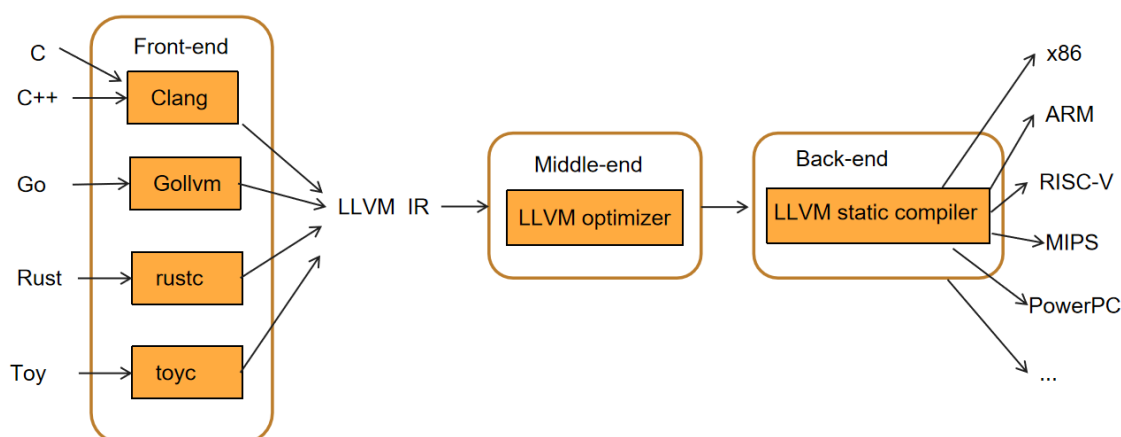


图 2.2 LLVM 的三阶段设计

LLVM 中间表示（Intermediate Representation, IR）是 LLVM 设计的关键部分，LLVM IR 是一种类似 RISC（精简指令集计算机）的三地址指令集，支持如加法、减法和控制等简单指令。LLVM 使用基于静态单赋值（Static Single Assignment, SSA）的中间表示，静态单赋值是编译器中间表示中非常重要的概念，它要求程序中的每个变量有且只有一个赋值语句。这样同名的变量就只会有相同的值，变量的引用指向唯一的定义，就不需要考虑寄存器的分配问题。

LLVM IR 作为编译器前端和后端的“桥梁”，非常适用于静态编译器的三阶段设计，通用性非常强，本文选用 LLVM IR 而非实际的高级语言作为研究基础有以下优势：

- (1) 新兴的现代化编译器框架 LLVM 前端已经支持 Haskell、Rust、C、C++ 及 Golang 等

高级语言。

(2) LLVM IR 仅包含约 50 个虚拟指令和约 30 个操作码，并使用 SSA 结构表示来简化数据流分析，且可以为每个函数都显示生成控制流图。LLVM 还可以在编译时将前端语言的一些信息比如源代码行号等写入 LLVM IR 的元数据中，这些元数据可以被用来将 IR 指令与前端实际源码进行对应，将对 LLVM IR 进行分析的结果转换为对前端语言的分析结果。

2.7 本章小结

本章介绍了后续章节研究漏洞检测需要用到的相关知识，首先介绍了深度学习相关的模型，如卷积神经网络，Transformer 模型，然后介绍了文本表示技术，分别从 One-hot 编码、词袋模型、TF-IDF、词嵌入模型四个方面详细介绍，然后介绍了程序切片、数据流分析、LLVM IR 相关背景知识，为后续的研究提供了理论依据。

第三章 基于静态切片语义与度量的代码漏洞检测

本章基于静态切片语义与度量代码的漏洞检测方法，能否从源代码中提取和缺陷相关的特征，是决定代码漏洞检测效果的关键因素。针对现有基于程序切片的漏洞检测方法仅获取代码语义信息的情况，存在可能无法全面准确地捕捉到潜在的漏洞的问题，本章提出了在 LLVM IR 层面的静态切片语义与度量代码的漏洞检测方法。代码的粒度大小起到了至关重要的作用，常见的粒度是文件级、函数级、行级别，文件级别的粒度太大，容易学习到和缺陷无关的特征，函数级别和行级没有考虑相互之间的依赖关系，而本章使用的程序切片级别的粒度，可以考虑到代码之间的数据依赖和控制依赖，具有丰富的语义特征。本章首先举例缓冲区溢出漏洞、空指针解引用漏洞、SQL 注入攻击漏洞、函数调用漏洞四种类型在 LLVM IR 层面兴趣点选取规则，并提出了基于 LLVM IR 缺陷的兴趣点检测算法，LLVM IR 作为一种中间表示语言，具有广泛的适用性。通过 SymPas 可以获取兴趣点的前向切片和后向切片，用 Inst2vec 获取 IR 指令的向量表示，并融合指令级切片度量，度量提供了对切片的详细度量信息。通过紧密度、重叠度、覆盖度和认知复杂度度量指标，可以深入分析切片之间的关系和特征。这使得漏洞检测方法能够更准确地评估切片的质量和相关性，从而提高检测的准确性和可靠性，最终通过混合模型 ResCNN-GRU 进行训练。

3.1 相关工具和总体框架

3.1.1 程序切片工具

程序切片技术有多种实现方法，包括基于数据流分析、控制流分析和语义分析等。程序切片可以用于不同的应用场景，如程序理解、调试、测试、维护等。目前有很多程序切片工具可供选择，以下是一些常见的程序切片工具，CodeSurfer 是一个商业化的静态分析工具，它可以生成程序的控制流和数据流图，并且可以用于程序切片和调试。JavaSlicer 是一个基于 Java 的程序切片工具，它可以通过分析程序的字节码来生成程序的切片^[1]。SlicingTool 是一个基于 C/C++ 的程序切片工具，它使用基于数据流分析的算法来生成程序的切片，并且支持多种语言和编译器。SliceFinder 是一个基于 Python 的程序切片工具，它可以生成 Python 程序的切片。除了以上的切片工具外，还有很多其他工具。本章使用的是一种基于轻量级上下文敏感依赖分析的符号化程序切片方法 SymPas (Symbolic Program Slicing)^[39]，SymPas 可以通过 LLVM 中间码 IR 实现过程间切片，无需构造易爆炸的 SDG 图或超图，切片的时间和空间成

本大大降低，对大型程序进行切片的效率更高。

```
#include <stdio.h>
int main() {
    int n, i, sum;
    void A(int *x, int *y);
    printf("Enter a positive number: ");
    scanf("%d", &n);
    sum = 0;
    i = 1;
    while (i <= n) {
        A(&sum, &i);
    }
    printf("sum = %d\n", sum);
    printf("i = %d\n", i);
    return 0;
}

void A(int *x, int *y) {
    void add(int *a, int *b);
    void inc(int *z);
    add(x, y);
    inc(y);
}

void add(int *a, int *b) {
    *a = *a + *b;
}

void inc(int *z) {
    int tmp = 1;
    add(z, &tmp);
}
```

图 3.1 sum3.c 示例

如图 3.1 所示，sum3.c 是一个经典的程序切片示例，目的是计算一个输入的正整数 n 以内的所有正整数的和。该程序包含了一个名为 A 的函数，该函数接受两个指向整数的指针作为参数，然后将它们分别传递给两个名为 add 和 inc 的函数，以执行加法和自增操作。主函数使用 $while$ 循环调用 A 函数，直到变量 i 大于输入的正整数 n 。最后，主函数打印变量 sum 和 i 的值。如图 3.2 所示，是对变量 i 的后向切片语句集，程序的与变量 i 的一部分被单独提取出来进行分析，使得代码更加清晰、易于理解和维护。程序切片可以使得程序员更容易发现代码中的缺陷和错误，并提供可行的解决方案，以便更好地维护和改进代码。

```
scanf("%d", &n);
i = 1;
while (i <= n) {
    A(&sum, &i);
}
```

```
}  
inc(y);  
*a = *a + *b;  
int tmp = 1;  
add(z, &tmp);
```

图 3.2 变量 i 的后向切片

3.1.2 总体框架

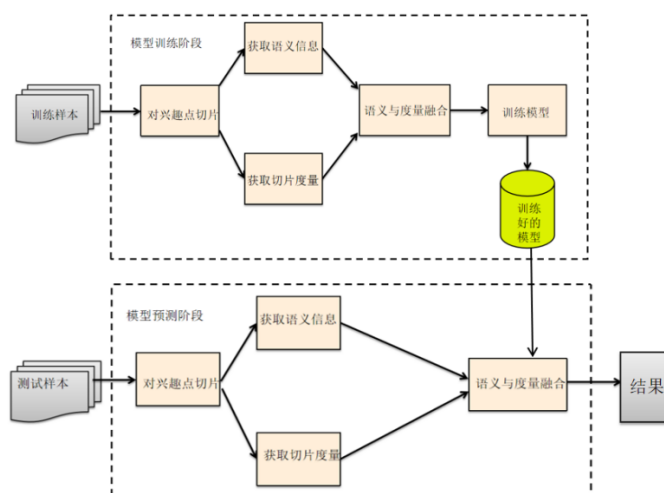


图 3.3 总体框架图

基于静态切片语义与度量的代码漏洞检测的总体框架，如图 3.3 所示，该框架主要包括两个阶段，模型训练阶段和模型预测阶段。

模型训练阶段：

(1) 训练样本转化为 LLVM IR：将训练样本的源代码转化为 LLVM IR 表示，以便后续分析和处理。

(2) 兴趣点选取：根据定义的兴趣点选取算法，在 LLVM IR 层面选择与缺陷相关的兴趣点。

(3) 获取切片语义：使用 SymPas 工具获取兴趣点的前向切片和后向切片，这些切片包含了与兴趣点相关的代码片段。同时，使用 Inst2vec 技术将 LLVM IR 指令转化为向量表示，捕捉其语义信息。

(4) 融合语义与度量：结合切片语义和度量信息，包括紧密度、重叠度、覆盖度和认知复杂度，将语义信息和度量信息进行融合。

(5) 混合模型训练：采用混合模型 ResCNN-GRU 进行训练，该模型结合了卷积神经网络（CNN）和门控循环单元（GRU）两种不同的神经网络结构，并利用残差结构进行连接。

这样的混合模型能够更好地捕捉代码的局部特征和序列信息，提高漏洞检测的准确性。

模型预测阶段：

(1) 测试样本转化为 LLVM IR：将待测试样本的源代码转化为 LLVM IR 表示。

(2) 兴趣点选取：根据定义的兴趣点选取算法，在 LLVM IR 层面选择与缺陷相关的兴趣点。

(3) 获取切片语义：使用 SymPas 工具获取兴趣点的前向切片和后向切片，这些切片包含了与兴趣点相关的代码片段。同时，使用 Inst2vec 技术将 LLVM IR 指令转化为向量表示，捕捉其语义信息。

(4) 融合语义与度量：结合切片语义和度量信息，包括紧密度、重叠度、覆盖度和认知复杂度，将语义信息和度量信息进行融合。

(5) 混合模型预测：使用训练好的混合模型 ResCNN-GRU 对测试样本进行预测，并输出相应的预测结果。

3.2 切片兴趣点

对代码进行程序切片需要以某个变量作为切点，进而获得其前向切片或后向切片。这个变量就是一个切入点，可以提取代码的一般特征，尤其是与漏洞有关的代码片段，需要找到“程序关注点”或“程序敏感点”的概念^[1]，关注点并不是人为设定的漏洞特征。从直观上，它可以被看作一个漏洞的“中心”，或者暗示存在漏洞的代码片段，这里称为兴趣点。在静态分析中，切入点可以是变量、函数参数、特定的语句或其他程序结构。通过选择与特定漏洞类型相关的切入点，可以聚焦于与该漏洞相关的代码片段，并提取有关漏洞的特征。兴趣点是根据代码漏洞特点来选择的，它可以是与漏洞直接相关的变量或语句，也可以是漏洞可能影响的程序区域。通过选择合适的兴趣点，可以捕捉到潜在的漏洞代码，并从中提取有关漏洞的信息。因此，兴趣点在基于静态切片语义与度量代码的漏洞检测方法中起着关键作用。可以更好地提取与漏洞相关的代码片段，用来进一步进行漏洞检测和分析。

3.2.1 缓冲区溢出相关漏洞

缓冲区溢出漏洞是一种常见的安全漏洞，它发生在程序试图将数据写入缓冲区时，写入的数据超出了缓冲区预先分配的内存空间，导致数据覆盖了相邻的内存区域。攻击者可以利用这个漏洞，向缓冲区中注入恶意代码或修改程序的行为^[42]。

这种漏洞通常出现在使用 C 和 C++ 等语言编写的程序中，因为这些语言对内存的管理和

分配较为低级,需要手动进行内存管理。一些常见的缓冲区溢出漏洞包括堆栈溢出和堆溢出。为了防止这种漏洞,可以使用一些安全编程技术,例如正确的内存分配和使用,输入验证和边界检查等。如图 3.4 所示,这是 SARD 中一个实例编号为 CVE-2010-0116 的缓冲区溢出的漏洞,这是在 OpenBSD 系统中的 `strtok` 函数中存在缓冲区溢出漏洞,攻击者可以利用该漏洞执行任意代码或者导致拒绝服务攻击,具有相当严重的危害。导致这个漏洞的原因,是在处理字符串时,没有对 `strtok()` 这个函数输入字符串的长度进行正确的检查,导致攻击者可以通过输入过长的字符串来覆盖程序堆栈中的其他数据。修复该缓冲区漏洞的方法是在调用 `strtok()` 函数之前,先对输入字符串的长度进行检查,并将输入字符串长度限制在合理的范围内。对应的 LLVM IR 指令如图 3.5,其中关键的部分是对输入字符串长度的检查,即以下代码: `%2 = getelementptr inbounds i8*, i8** %argv, i32 1`,这条指令计算出了输入字符串的指针,并将其存储在 `%3` 变量中。它的含义是从输入参数中获取第二个参数的地址 (`i8** %2`),该地址存储了输入字符串的指针,即输入字符串的地址。`getelementptr inbounds` 指令将 `%2` 指针向前偏移 0 个元素,相当于得到了 `%2` 指向的地址的值,即输入字符串的指针。该指令是在对输入参数的检查之后执行的,以确保第二个参数存在 (`%1 = icmp slt i32 %argv, 2`) 并且输入字符串的长度小于 10 (`%5 = icmp ult i64 %4, 10`)。如果输入字符串长度大于等于 10,则该指令将导致缓冲区溢出,因为在后续的 `strcpy` 和 `strtok` 调用中, `%input` 变量将被填充超过其分配的大小,导致内存越界。因此,输入字符串的长度检查是必要的,以避免缓冲区溢出漏洞的发生。

```
#include <stdio.h>
#include <string.h>
int main(int argc, char* argv[]){
    char input[10];
    char *token;
    if (argc < 2){
        printf("Usage: %s <input_string>\n", argv[0]);
        return 1;
    }
    strcpy(input, argv[1]);
    token = strtok(input, "");
    while (token != NULL){
        printf("%s\n", token);
        token = strtok(NULL, "");
    }
    return 0;
}
```

图 3.4 缓冲区溢出漏洞示例

```

define i32 @main(i32 %argc, i8** %argv){
    %input = alloca [10 x i8]
    %token = alloca i8*
    %1 = icmp slt i32 %argc, 2
    br i1 %1, label %exit, label %check_input
check_input:
    %2 = getelementptr inbounds i8*, i8** %argv, i32 1
    %3 = load i8*, i8** %2
    %4 = call i64 @strlen(i8* %3)
    %5 = icmp ult i64 %4, 10
    br i1 %5, label %strtok, label %exit
strtok:
    %6 = getelementptr inbounds [10 x i8], [10 x i8]* %input,
    , i32 0, i32 0
    %7 = getelementptr inbounds i8*, i8** %argv, i32 1
    %8 = load i8*, i8** %7
    call void @strcpy(i8* %6, i8* %8)
    %9 = getelementptr inbounds [10 x i8], [10 x i8]* %input, i32 0, i32 0
    %10 = load i8*, i8** %9
    %11 = call i8* @strtok(i8* %10, i8* getelementptr
    inbounds ([2 x i8], [2 x i8]* @.str, i32 0, i32 0))
    store i8* %11, i8** %token
    %12 = icmp ne i8* %11, null
    br i1 %12, label %loop, label %exit
loop:
    %13 = load i8*, i8** %token
    %14 = call i32 (i8*, ...) @printf(i8* getelementptr
    inbounds ([3 x i8], [3 x i8]* @.str.1, i32 0, i32 0), i8* %13)
    %15 = getelementptr inbounds i8*, i8** %token, i32 0
    %16 = load i8*, i8** %15
    %17 = call i8* @strtok(i8* null, i8* getelementptr
    inbounds ([2 x i8], [2 x i8]* @.str, i32 0, i32 0))
    store i8* %17, i8** %token
    %18 = icmp ne i8* %17, null
    br i1 %18, label %loop, label %exit
exit:
    ret i32 0
}

```

图 3.5 缓冲区溢出漏洞示例的 IR 指令

3.2.2 空指针解引用漏洞

空指针解引用（Null Pointer Dereference）指的是在程序中对空指针进行了解引用操作，从而导致程序崩溃或者其他异常行为。这种漏洞通常发生在程序员没有进行足够的空指针检

查，或者在空指针检查的情况下仍然对空指针进行了解引用的情况下^[43]。空指针解引用漏洞已经成为许多重大软件漏洞的原因之一。如图 3.6 所示，是 SARD 中的一个空指针解引用的 C 语言漏洞实例，漏洞名称为 SARD CVE-2011-2895。这个漏洞在某些情况下，OpenSSL 1.0.0e 及更早版本中的 X509 证书验证功能可能会发生空指针解引用漏洞，攻击者可能会利用此漏洞造成拒绝服务攻击或可能的执行任意代码。

```
int verify_cb(int ok, X509_STORE_CTX *ctx)
{
    char buf[256];
    X509 *err_cert;
    int depth, err;
    SSL *ssl;
    ssl = X509_STORE_CTX_get_ex_data(ctx, SSL_get_ex_data_X509_STORE_CTX_idx());
    if (ssl)
        SSL_get0_peer_scts(ssl, NULL, 0);
    //...
}
```

图 3.6 空指针解引用漏洞示例

这个漏洞原因是在函数 VerifyX509Certificate()中，如果证书链中有一个证书没有 subjectName 字段，则不会为该证书创建 X509_NAME 对象，但是在后面的代码中会尝试使用这个未初始化的指针来调用 X509_NAME_cmp()函数，从而导致了空指针解引用漏洞。

解决该漏洞的方法是，在调用 X509_NAME_cmp()函数之前，先判断指针是否为空，如果为空则不应该调用该函数，可以返回一个错误码。同时，也可以在创建 X509_NAME 对象时，添加对 subjectName 字段是否存在的判断，如果不存在，则不创建该对象。

```
define i32 @verify_cb(i32 %ok, %X509_STORE_CTX* %ctx) {
entry:
    %ssl = call i8* @X509_STORE_CTX_get_ex_data(%X509_STORE_CTX* %ctx, i32 69)
    %cmp = icmp ne i8* %ssl, null
    br i1 %cmp, label %if.then, label %if.end
if.then:
    %null = alloca i8*, align 8
    %num_null = alloca i32, align 4
    store i8* null, i8** %null, align 8
    store i32 0, i32* %num_null, align 4
    call void @SSL_get0_peer_scts(i8* %ssl, i8** %null, i32* %num_null)
    //...
    br label %if.end
if.end:
    //...
}
```

图 3.7 空指针解引用漏洞对应 IR 指令

如图 3.7 所示, 在 LLVM IR 指令中, 可以看到 icmp 指令比较了 X509_STORE_CTX_get_ex_data 函数返回的指针和空指针。如果指针不为空, if.then 块将被执行, 其中 SSL_get0_peer_scts 函数对该指针进行了解引用, 导致了空指针解引用漏洞。

3.2.3 SQL 注入漏洞

SQL 注入攻击是一种通过 Web 应用程序将恶意 SQL 代码注入到数据库引擎中的攻击方式。攻击者可以利用这种漏洞来绕过认证、获取敏感信息、篡改数据等^[44]。SQL 注入漏洞在 Web 应用程序中很常见, 是 OWASP 十大漏洞中的第一名。攻击者通过 Web 应用程序向后端数据库引擎发送恶意 SQL 语句, 使得数据库执行这些语句并返回结果。攻击者可以通过构造 SQL 语句来实现不同的攻击目的, 比如绕过认证、获取敏感信息、篡改数据等。常见的 SQL 注入攻击方式包括: 盲注、联合查询注入、错误信息注入、布尔盲注、时间盲注等。

对于 Web 应用程序, 最好的防御措施是输入验证和参数化查询。输入验证可以防止攻击者通过输入特殊字符来欺骗 Web 应用程序, 参数化查询可以防止攻击者将恶意 SQL 代码注入到查询中。同时, 还可以通过漏洞扫描工具来检测 Web 应用程序是否存在 SQL 注入漏洞。常见的漏洞扫描工具包括: Burp Suite、Nessus、OpenVAS 等。为了防止 SQL 注入攻击, 需要采取以下防御措施, 输入验证: 对于用户输入的数据进行验证, 限制输入的字符类型和长度, 防止输入恶意字符; 参数化查询: 使用预编译语句, 将输入的参数与 SQL 语句分开处理, 防止攻击者注入恶意 SQL 代码; 最小权限原则: 在数据库中创建专门的用户, 只给予最小的权限, 防止攻击者利用 SQL 注入攻击获取高权限账户的密码; 安全编码实践: 编写安全的代码, 包括使用编码函数 (如 htmlspecialchars、mysql_real_escape_string 等) 对数据进行编码等。

如图 3.8 所示, 一个 SQL 注入漏洞的 C 代码示例, 该代码存在 SQL 注入漏洞, 攻击者可以在 id 参数中输入恶意 SQL 代码并导致查询执行错误。在第 15 行中, id 变量是从命令行参数中获取的, 而后直接将其拼接到 21 行中的 SQL 语句, 这种拼接 SQL 语句的方式很容易受到攻击者的注入攻击。例如, 攻击者可以将命令行参数设置为以下值: 1; DROP TABLE table1; 这将导致 SQL 查询变为: SELECT * FROM table1 WHERE id=1; DROP TABLE table1; 这样攻击者就可以通过注入攻击来删除整个表。为了防止 SQL 注入攻击, 应该避免直接拼接 SQL 语句, 而是使用参数化查询或使用数据库 API 中的转义函数来转义输入的参数。例如使用 `char *id = mysql_real_escape_string(&mysql, argv[1], strlen(argv[1]));` 这样就能够正确地处理输入中的特殊字符并避免 SQL 注入攻击。将这个 SQL 注入漏洞的 C 代码示例转成

LLVM IR 指令如图 3.9, 和 sql 注入相关的关键语句是第九行, `call i32 @sprintf(i8* %query, i8* getelementptr inbounds ([26 x i8], [26 x i8]* @.str, i32 0, i32 0), i8* %table, i8* %id)`, 其中, %query 是用于存储 SQL 查询字符串的内存缓冲区, %table 和 %id 分别是表名和 ID 参数。第二个参数是查询字符串的格式化模板, 它使用了 %s 标记来表示字符串参数的位置。由于该模板字符串是程序内部硬编码的, 因此容易受到注入攻击。

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <mysql.h>
int main(int argc, char* argv[])
{
    MYSQL mysql;
    MYSQL_RES *result;
    MYSQL_ROW row;
    char query[100];
    char *user = "username";
    char *pass = "password";
    char *db = "dbname";
    char *table = "table1";
    char *id = argv[1];
    mysql_init(&mysql);
    if (!mysql_real_connect(&mysql, "localhost", user, pass, db, 0, NULL, 0)) {
        printf("Error connecting to database: %s\n", mysql_error(&mysql));
        return 1;
    }
    sprintf(query, "SELECT * FROM %s WHERE id=%s", table, id);
    if (mysql_real_query(&mysql, query, strlen(query))) {
        printf("Error querying database: %s\n", mysql_error(&mysql));
        mysql_close(&mysql);
        return 1;
    }
    result = mysql_store_result(&mysql);
    while ((row = mysql_fetch_row(result))) {
        printf("%s %s %s\n", row[0], row[1], row[2]);
    }
    mysql_free_result(result);
    mysql_close(&mysql);
    return 0;
}
```

图 3.8 SQL 注入漏洞示例

```
@.str = private unnamed_addr constant [26 x i8]c"SELECT
FROM %s WHERE id=%s\00", align 1
define i32 @main(i32 %argc, i8** %argv){
```

```
entry:
%1 = getelementptr inbounds i8*, i8** %argv, i32 1
%table = load i8*, i8** %1, align 8
%2 = getelementptr inbounds i8*, i8** %argv, i32 2
%id = load i8*, i8** %2, align 8
%query = alloca [128 x i8], align 1
call i32 @sprintf(i8* %query, i8* getelementptr inbounds
([26 x i8], [26 x i8]* @.str, i32 0, i32 0), i8* %table, i8* %id)
...
}
```

图 3.9 SQL 注入漏洞示例的 IR 指令

3.2.4 函数调用漏洞

函数调用漏洞指的是在程序中存在对函数调用的错误或不安全使用，导致程序在运行时出现漏洞或安全问题^[45]。常见的函数调用漏洞包括以下几种：调用不安全函数导致的漏洞，某些函数在使用时需要谨慎，如 `strcpy`、`strcat`、`gets` 等，因为它们没有进行足够的输入检查，容易被攻击者利用进行缓冲区溢出攻击。参数传递错误导致的漏洞，在函数调用时，如果参数传递错误，比如传递了错误的参数类型、数量或顺序，就有可能导致程序崩溃或出现漏洞。函数返回值未检查导致的漏洞，如果函数返回值未被检查，就可能导致一些安全问题，比如函数返回的指针被用于后续的操作，但是实际上它指向了一个无效的内存地址。函数指针调用导致的漏洞，在使用函数指针时，如果没有进行足够的检查，就有可能被攻击者利用进行代码注入攻击。未初始化的变量所引起的漏洞：如果程序中存在未初始化的变量，就可能导致函数调用时出现未知行为，从而引发漏洞。函数调用漏洞在软件安全中是非常常见的，攻击者可以通过利用这些漏洞，对系统进行各种攻击，比如执行恶意代码、窃取敏感信息、拒绝服务等。因此，在开发软件时，需要特别注意函数调用的安全使用，并进行必要的检查和过滤。

本节总结了常见的不安全函数，可能存在许多函数的调用会带来漏洞风险。以下是一些常见的函数调用，可能会导致漏洞。内存分配函数：如 `malloc`、`calloc`、`realloc` 等。在使用这些函数时，必须小心处理内存分配失败的情况，以及确保已分配的内存大小正确，以避免缓冲区溢出漏洞等问题。字符串处理函数：如 `strcpy`、`strcat`、`sprintf`、`vsprintf` 等。在使用这些函数时，必须确保目标缓冲区具有足够的空间来存储要复制或附加的数据，否则可能会导致缓冲区溢出漏洞。文件操作函数：如 `fopen`、`fclose`、`fread`、`fwrite` 等。在使用这些函数时，必须检查文件打开和关闭操作是否成功，以及确保读取和写入的数据量正确，以避免文件操作漏洞。网络通信函数：如 `socket`、`bind`、`listen`、`accept`、`recv`、`send` 等。在使用这些函数时，

必须注意网络安全问题，包括输入验证、数据加密、数据过滤等，以避免网络攻击漏洞。时间函数：如 `time`、`gettimeofday`、`clock` 等。在使用这些函数时，必须确保在使用之前已正确初始化，并正确处理时区、闰秒等问题，以避免时间相关的漏洞。密码学函数：如 `rand`、`srand`、`crypt`、`openssl` 等。在使用这些函数时，必须确保在安全环境中使用，并注意使用加密算法、密钥管理等，以避免密码学漏洞。

3.2.5 基于 LLVM IR 的漏洞兴趣点检测算法

本节对常见漏洞类型归纳总结，内存错误（Memory Error）中如：缓冲区溢出栈溢出，资源泄漏，空指针解引用等。输入验证和表示（Input Validation and Representation）中如：SQL 注入，格式字符串等，函数调用（Function Calls）中如：使用错误的函数，重复释放内存，使用过时函数，标准函数错误使用，使用不安全函数等总结出相关四种类型的漏洞，同时针对这四种类型总结了兴趣点选取的准则：

（1）数组变量使用：数组使用中常见的漏洞有越界写入、越界读取等。在读取或写入时，由于对变量没有进行完整性检查从而导致越界的存在。数组的操作往往与 `alloca`、`load`、`store` 等指令相关，用来声明、赋值或存储数组，操作数没有进行边界检查便可能会导致程序错误，数组的大小、数组下标、数组元素的类型等作为兴趣点，可以使用 `getelementptr` 指令计算数组元素的地址，使用 `load` 和 `store` 指令进行读写操作。

（2）指针变量使用：常见漏洞有空指针解引用、内存泄漏、Use-After-Free 等。在使用指针时，指针为空时的引用，或者是疏忽或错误造成指针未能释放，以及指针释放后仍使用，都会造成程序的异常。操作指针通常使用 `getElementPtr` 指令，空指针解引用漏洞的兴趣点：指针变量、指针是否为空、指针解引用的偏移量等，可以使用 `load` 和 `store` 指令进行读写操作，使用 `getelementptr` 指令计算指针解引用的地址。

（3）算术表达式：算术表达式中变量的不正确赋值或操作，会造成比如整数溢出、除零错等问题。该类漏洞通常与 `add`、`sub`、`mul` 和 `div` 等算术运算指令相关。

（4）API/库函数调用：如 `strcpy`、`strcat`、`sprintf`、`gets`、`scanf` 等库函数调用，这些函数在对缓冲区进行操作时，不执行边界检查，很容易导致缓冲区溢出，从而导致程序崩溃。文件操作函数：如 `fopen`、`fclose`、`fread`、`fwrite` 等。在使用这些函数时，必须检查文件打开和关闭操作是否成功，以及确保读取和写入的数据量正确，以避免文件操作漏洞。网络通信函数：如 `socket`、`bind`、`listen`、`accept`、`recv`、`send` 等，这些函数都可能存在风险。首先，在 LLVM IR 中函数调用使用 `call` 指令；对于具体的函数调用，比如库函数 `strcpy` 调用，可能会造成的

缓冲区溢出通常与 Load、Store、GetElementPtr 和 Alloca 等指令类型相关，Load 和 Store 指令用于读取和写入内存，GetElementPtr 指令用于计算指向数组元素的指针，而 Alloca 指令用于在堆栈上分配内存。当这些指令被用于缓冲区的边界检查不正确时，就可能导致缓冲区溢出漏洞。并针对以上四种类型总结了兴趣点选取的准则中的指令，给出了指令中选取兴趣点的依据。

(1) 对于 alloca 指令：alloca 指令的参数和返回的指针可以作为兴趣点。如果 alloca 指令的参数是一个全局变量，则返回的指针可以作为兴趣点；如果 alloca 指令的返回值是一个函数的返回值，则返回的指针可以作为兴趣点；如果 alloca 指令的参数是其他指针，则返回的指针可以作为兴趣点；

(2) 对 GEP 指令 (getElementPtr)：getElementPtr 操作的指针变量选为兴趣点指针参数：如果 GEP 指令的第一个操作数是指针类型，则该指针所指向的内存地址中包含的数据可以作为兴趣点。常量偏移量：如果 GEP 指令的第二个操作数是常量，则该常量可以作为兴趣点。变量偏移量：如果 GEP 指令的第二个操作数是变量，则该变量的值可以作为兴趣点。结构体成员访问：如果 GEP 指令的第二个操作数是一个索引序列，用于访问结构体的特定成员，则该索引序列可以作为兴趣点。数组访问：如果 GEP 指令用于访问数组元素，则该索引序列可以作为兴趣点；

(3) 赋值指令 (store 和 load)：对于 store 指令，将其存储的值对应的地址标记为兴趣点；对于 load 指令，将其加载的地址对应的值标记为兴趣点。

(4) 对于调用指令 (call)，将其指令中调用的参数标记为兴趣点，如果函数返回值为指针类型，则将其标记为兴趣点；

(5) 对于算术指令 (add、sub、mul、div 等)，将其任何一个操作数标记为兴趣点。

(6) 对于比较指令 (icmp 和 fcmp)：将其任何一个操作数标记为兴趣点。

基于 LLVM IR 的漏洞兴趣点检测算法如算法 3.1 所示，首先将程序 P 通过 LLVM 的 Clang 前端编译器转化为中间代码表示，即 LLVM IR 指令。然后，利用 LLVM IR 指令创建一个函数调用图，用于记录函数之间的调用关系。之后，创建了一个空的兴趣点集合 K。通过遍历函数调用图中的每个函数和函数内的基本块，以及基本块内的每条 LLVM IR 指令，对指令进行六大兴趣点准则的判断。如果指令满足兴趣点准则，就从切片表中获取与该指令相关的变量，并将这些变量添加到兴趣点集合 K 中。最终，返回兴趣点集合 K。使用 LLVM IR 指令和函数调用图等中间表示来抽象底层的程序细节，使得分析过程更加高层次和抽象化。这种抽象层次的提高使得分析过程更容易理解和推理，减少了对具体编程语言细节的依赖。总之，使用 LLVM IR 中间表示来找兴趣点具有平台无关性、丰富的信息、可扩展性和中间表示优化

等优势,这使得基于 LLVM IR 进行静态分析能够更加准确、全面和可靠地找到程序中的兴趣点。

算法 3.1: 基于 LLVM IR 的漏洞兴趣点检测算法

输入: 程序 P

输出: 兴趣点集合 K

```
1.   $llvm\_ir \leftarrow compile\_to\_llvm\_ir(P)$  #将程序  $P$  通过编译器 LLVM 的 Clang 转化为 LLVM IR 指令.
2.   $call\_graph \leftarrow create\_call\_graph(llvm\_ir)$  #创建 IR 指令的函数调用图  $call\_graph$ .
3.   $K \leftarrow \{\}$  #初始化空的兴趣点集合  $K$ 
4.  For  $function$  in  $call\_graph.functions$  do #遍历函数调用图中的每个函数  $function$ 
5.      For  $block$  in  $function.basic\_blocks$  do #遍历函数  $function$  中的每个基本块  $block$ 
6.          For  $instruction$  in  $block.instructions$  do #遍历基本块  $block$  中的每条 IR 指令
7.              If  $satisfies\_interesting\_point\_criteria(instruction)$  then #判断该指令是否满足六大兴趣点
8.                  #从切片表中获取与该指令相关的变量,并将其添加到兴趣点集合  $K$  中
9.                   $variables \leftarrow get\_related\_variables(instruction)$ 
10.                  $K.update(variables)$ 
11.             End if
12.         End for
13.     End for
14. End for
15. Return  $K$ 
```

3.3 提取语义信息

为了获取切片语句的语义信息,需要将代码片段转换为向量表示,然后通过深度学习模型提取更高级的特征表示。深度学习模型能够自动学习和提取代码片段中的抽象特征,从而更好地捕捉代码的语义和行为。这使得模型具有更强的表达能力,能够提高对代码漏洞和缺陷的检测能力。

图 3.10 所示,函数是 fund 函数,它定义了三个字符数组变量: data、dataBuffer 和 source。它将 data 指针设置为 dataBuffer 数组之前 8 个字节的位置,然后使用 memmove 函数将 source 数组的内容复制到 data 指针指向的内存位置,长度为 100 个字符。由于 data 指针指向了 dataBuffer 数组之前的一些内存,因此该操作会覆盖一些未定义的内存。最后,它调用 printDate 函数,将 data 指针指向的字符串作为参数传递给它。以第 22 行代码为例,指针数组可能会发生越界的问题,那么 data 这个变量就是一个兴趣点,考虑到它的上下文的控制依赖信息和数据依赖信息,通过 SymPas 可以获取变量的前向切片和后向切片,如表 3.1 静态切片表所示。将前向切片和后向切片的行号合并,按照行号从小到大的顺序将代码合并起来,如图 3.11 所示 data 变量所对应的 IR 指令切片。

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  void printDate(const char * d){
6      if(d!=NULL)
7          printf("%s\n",d);
8  }
9  void fund(){
10     char *data;
11     char dataBuffer[100];
12     char source[100];
13     memset(dataBuffer,'A',99);
14     dataBuffer[99]='\0';
15     while(1){
16         data = dataBuffer - 8;
17         break;
18     }
19     memset(source,'C',99);
20     source[99]='\0';
21     memmove(data,source,100*sizeof(char));
22     data[99]='\0';
23     printDate(data);
24 }
25 int main(void) {
26     fund();
27     return 0;
28 }

```

图 3.10 示例代码

```

label %6,%3 = icmp ne i8* %0 , null,
br i8* %3 , label %4 ,
%5 = call i32(i8*, ...)* @printf ( [4 x i8] [4 x i8] getelementptr inbounds
( [4 x i8]* @.str , i64 0, i64 0 ), i8* %0 )
br label %6,
%1 = alloca i8* , align 8
%2 = alloca [100 x i8] , align 16,
%3 = alloca [100 x i8] , align 16,
call void(metadata, metadata, metadata)* @llvm.dbg.declare ( i8** %1,
metadata* %data, metadata c"!28" ),
call void(metadata, metadata, metadata)* @llvm.dbg.declare ( [100 x i8]* %3, meta
data* %source, metadata c"!28" ),
%4 = getelementptr inbounds [100 x i8]* %2 , i64 0, i64 0,
call void(i8*, i8, i64, i1)* @llvm.memset.p0i8.i64 ( [100 x i8]* %4, i64 65, i64
99, i8 0 ),

```

```
.....
%9 = getelementptr inbounds i8* %6 , i64 99,
store i64 0 , i8* %9 , align 1,
call void(i8*)* @printDate ( i8* %6 ),
```

图 3.11 data 变量 IR 指令切片

表 3.1 静态切片表

变量	后向切片	前向切片
d@printDate	{6, 13, 14, 16, 19, 20, 21, 22, 23, 26}	{6, 7}
data@fund	{10, 11, 12, 13, 14, 15, 16, 19, 20, 21, 22}	{23, 6, 7}
dataBuffer@fund	{11, 12, 13, 14}	{16, 21, 22, 23, 6, 7}
source@fund	{12, 19, 20}	{21, 22, 23, 6, 7}

为了获取 IR 切片指令集的嵌入表示，借鉴 Inst2vec 工具^[46]中提出的思想。首先，对每个指令进行预处理，将其转化为更通用的表示形式。预处理的目的是提取具有代表性的语义信息，并剔除对于预测漏洞不太重要的信息。随后，将经过预处理的指令表示输入到 word2vec 的 skip-gram 模型^[34]中如图 3.12，以获得每个指令对应的向量嵌入。选择使用 skip-gram 模型的原因在于它能够通过相邻之间的指令推断出每个指令的语义含义，并将语义相关的指令在向量空间中保持近距离。

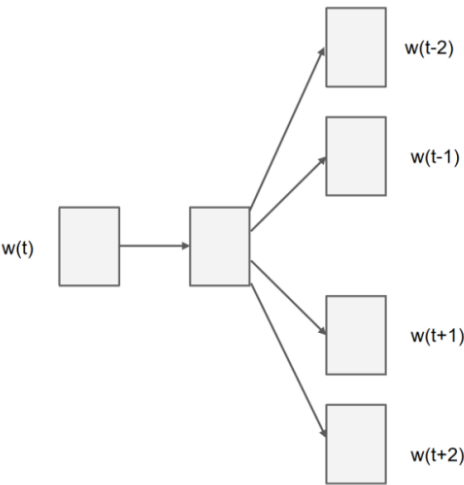


图 3.12 Skip-gram 模型结构

对于每个 IR 指令，需要消除一些与预测漏洞无关的信息。其中包括对齐和调试信息，因为这些信息通常与程序的语义含义无关，对于预测漏洞并不具有帮助。另外，变量名和标签被替换为通用的标记，如"%ID"和"%LABEL"。这样做是因为在指令的语义表示中，变量名和标签通常不是关键信息，用通用的标记代替可以减少指令表示的维度并去除不必要的细节。接下来，整数和浮点数的字面值被替换为"@INT VAL"和"@FLOAT VAL"。这是因为整数和浮点数字面值的范围很广，不可能为每个单独的值学习一个表示。相反，使用通用的标记表示这些字面值可以减少嵌入向量的维度，并将注意力集中在字面值的语义含义上。在预处理过程中，还对函数名称和函数类型进行了处理。对于同一文件中的函数，它们的名称被替换为

"%FUNC NAME"。这是为了进一步减少指令表示中的维度，将注意力集中在函数的功能而非具体名称上。对于不同文件中的函数，保留了它们的名称。这是因为这些函数往往是实用的 API 函数，要么是在项目本身中实现的，要么是经常在其他常用库中使用的。函数类型由抽象的函数参数组成，并被保留下来以区分不同的函数。保留函数类型的目的是在后续的指令嵌入生成过程中，当函数名称被抽象化时，仍然能够区分不同的函数。

表 3.2 预处理指令示例

IR 指令	预处理后的指令
store i32 %15, i32* %length left, align 4, !dbg !182	store i32 %ID, i32* %ID
br i1 %cmp27, label %if.end30, label %if.then29, !dbg !201	br i1 %ID, label %LABEL, label %LABEL
%ru6 ncts32 = bitcast %union.anon* %rip6un31 to [1 x %struct.netinfo6]*, !dbg !207	bitcast [@INT VAL x %STRUCT TYPE], %STRUCT TYPE* %ID
%call99=call i32 @rip6 entry print(%struct.netdissect options* %79, %struct.netinfo6* %80, i32 0), !dbg !268	Call i32(%STRUCT TYPE*, %STRUCT TYPE*, i32) %FUNC NAME
call void @llvm.memcpy.p0i8.p0i8.i64(i8* %11, i8* %13, i64 4, i32 2, i1 false), !dbg !474	call void (i8*, i8*, i64, i32, i1) llvm.memcpy.p0i8.p0i8.i64

Skip-gram 模型的核心思想是通过上下文来预测目标词语，模型试图学习一个嵌入空间，其中相似的词语在向量空间中距离更近。对于每个 IR 指令，我们将其视为目标词语，并考虑其周围的指令作为上下文。这些上下文指令可以是控制流、数据流或使用定义依赖关系的节点对，以及与目标指令存在控制流依赖关系的相邻指令。采用了一定的上下文范围，以捕捉指令之间的关联性。在训练阶段，将预处理后的 IR 指令表示作为输入，通过训练 Skip-gram 模型来学习指令的向量嵌入。模型的目标是最大化目标指令和其上下文之间的相关性。通过迭代优化模型参数，得到了每个指令在嵌入空间中的向量表示。Skip-gram 模型的优势在于它能够从大规模的数据中学习到丰富的语义信息。通过考虑指令的上下文，模型能够捕捉到指令之间的语义关联，使得具有相似操作数或执行结果的指令在向量空间中更加接近，这为获取 IR 指令嵌入表示提供了一种有效的方式。

3.4 提取切片度量特征

当提及软件度量时，我们通常关注的是软件的质量和可维护性等方面。在软件工程中，度量是一种广泛应用的方法，可以帮助开发者预测可能存在的风险，并评估软件质量。软件度量技术主要包括两大类：源代码度量和过程度量。源代码度量是通过对源代码进行分析和度量，来评估软件质量和软件开发过程中的各种属性的方法。源代码度量可以用于评估代码

的复杂性、可维护性、可读性、可靠性和可重用性等方面。过程度量是通过对软件开发过程进行分析和度量，来评估软件开发过程的各种属性和质量，可以用于评估软件开发过程的效率、质量、风险、复杂性和成本等方面。目前，有许多可以用来预测代码缺陷的度量方法和指标，包括以下几种，圈复杂度是一种用于度量代码复杂度的方法，较高的圈复杂度常常与代码缺陷密切相关，可以作为预测代码缺陷的重要指标之一。代码行数是另一种常用的度量代码复杂度的方法。通常情况下，较高的代码行数也与代码缺陷密切相关。因此，代码行数可以作为预测代码缺陷的重要指标之一。Halstead 度量是一种基于代码中操作符和操作数数量的度量方法，可以用来预测代码中可能出现的缺陷。坏味道代码指标是一种基于代码中坏味道（Bad Smell）的度量方法，坏味道通常表示代码中的一些不良设计和编码实践，这些不良实践往往会导致代码缺陷。常用的坏味道代码指标包括函数参数数量、代码复杂度、重复代码等。这些度量可以用于预测软件缺陷，提高软件的质量和可维护性。

程序切片是一种广泛应用于软件工程中的技术，可以将程序划分为更小、更易于理解和分析的部分，后来被广泛运用于软件模块的度量，如内聚度、耦合度、复杂性等问题的分析。已有的切片度量主要关注于基于程序切片的内聚性度量和耦合性度量。然而，指令级切片不仅可以得到更精确的度量结果，还可以减少不同编程语言对程序分析的影响。由于指令级切片的分析粒度更细，它可以提供更精确的度量结果，进而更准确地分析程序的复杂性^[47]。因此，指令级切片在软件度量中具有广泛的应用前景。

本章基于在 LLVM IR 指令构建了紧密度在 LLVM IR 指令上进行前向切片和后向切片构建紧密度公式、重叠度公式、覆盖度公式、可以全面的考虑程序的紧密度、重叠度、覆盖度情况。

$$IRFBTightness = \frac{|SL_{int}|}{len(M)} \quad (3.1)$$

$$IRFBCoverage = \frac{1}{|V|} \left(\sum_{x \in V_i} \frac{|FSL_x|}{len(M)} + \sum_{x \in V_o} \frac{|BSL_x|}{len(M)} \right) \quad (3.2)$$

$$IRFBOverlap = \frac{1}{|V|} \left(\sum_{x \in V_i} \frac{|SL_{int}|}{|FSL_x|} + \sum_{x \in V_o} \frac{|SL_{int}|}{|BSL_x|} \right) \quad (3.3)$$

上面公式中所述的 $len(M)$ ：切片模块 M 中 IR 语句的行数， V_i ：模块中输入变量的集合， i 为输入变量下标。 V_o ：模块中输出变量的集合， o 为输出变量的下标。 V ：模块中输入变量和输出变量的集合。 FSL_x ：变量 x 的前向切片。 BSL_x ：变量 x 的后向切片。 SL_{int} ： V_o 中所有

变量的后向切片和 V_i 中所有变量的前向切片的交集。此外本章基于 IR 指令的前向切片和后向切片构建新型的切片度量指标，基于指令切片的认知复杂度度量指一标切片认知域。

$$IRsliceSpatial = \frac{\sum_{i \in B} \sqrt{(x_i - x_k)^2 + (y_i - y_k)^2 + (z_i - z_k)^2}}{len(B)} \quad (3.4)$$

该度量的含义为，指令行中的距离越大，了解该切片的目的数据流信息和控制流信息所需的认知努力越多，存在缺陷的可能就越大。对程序的兴趣点进行前向切片和后向切片之后，会得到一定数量的切片指令，受到聚类的思想启发，这些切片指令集和兴趣点距离越大，越不容易聚合在一起，所需的认知努力越多，存在缺陷的可能性越大。为了精准的考虑距离，本文给每条指令提出了坐标的概念，对 IR 文件生成其函数调用关系图(Call Graph, CG)，CG 图上的节点是一个函数，通过从上到下，从左到右层次遍历给每个函数一个坐标，对每个函数生成其流程控制图(Control Flow Graph, CFG)，通过层次遍历给 CFG 中的每个 block 一个坐标，给每个 block 中的指令一个坐标。公式 3.4 中： $len(B)$ 是前向切片和后向切片的数量。 x_i ：切片中第 i 个切片在 CG 图上坐标， y_i ：切片中第 i 个切片在 CFG 图上坐标， z_i ：切片中第 i 个切片在 block 上坐标。 x_k ：兴趣点在 CG 图上坐标， y_k ：兴趣点在 CFG 图上坐标， z_k ：兴趣点在 block 上坐标。

3.5 深度学习模型构建

门控制神经网络 GRU 是由 Cho 等提出的^[48]，它是长短期记忆神经网络 LSTM 的一个变种，是为了解决长期记忆和反向传播中的梯度等问题。但是 GRU 结构更简单，便于计算，更易于实现。GRU 的详细内部结构如图 3.13。其中， x_t 是当前时刻的输入， $\tilde{h}_t$ 是当前时刻的候选输出， h_{t-1} 是上一时刻的输出也作为本时刻的输入， h_t 是当前时刻的输出。模型的表达式如下：

$$r_t = \sigma(W_r \cdot [x_t, h_{t-1}]) \quad (3.5)$$

$$z_t = \sigma(W_z \cdot [x_t, h_{t-1}]) \quad (3.6)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \quad (3.7)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (3.8)$$

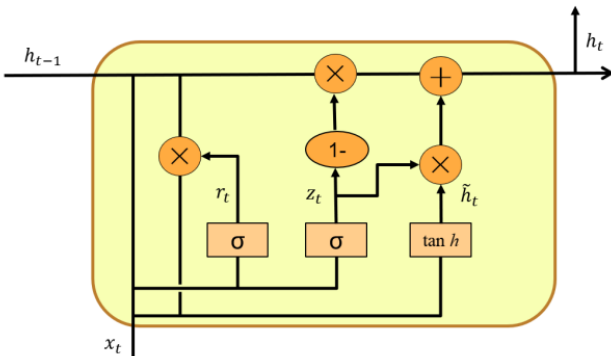


图 3.13 GRU 的详细内部结构

GRU 一共有两个门，它将 LSTM 中的输入门和遗忘门合并为更新门，控制前面记忆信息能够继续保留到当前时刻的数据量，或者说决定有多少前一时间步的信息和当前时间步的信息要被继续传递到未来；GRU 的另一个门称为重置门（Reset Gate），控制要遗忘多少过去的信息。LSTM 和 CRU 都是通过各种门函数来将重要特征保留下来，这样就保证了在 long-term 传播的时候也不会丢失，适合捕获长序列的特征，捕获长代码序列的依赖信息。此外 GRU 相对于 LSTM 少了一个门函数，因此在参数的数量上也是要少于 LSTM 的，所以整体上 GRU 的训练速度要快于 LSTM 的。卷积神经网络（CNN）是一种在深度学习领域广泛使用的多层神经网络。通过共享权值来解决随着神经网络层数的加深而带来的参数过多问题。最典型的卷积网络，由卷积层、池化层、全连接层组成。其中卷积层与池化层配合，组成多个卷积组，逐层提取特征，最终通过若干个全连接层完成分类。

残差网络(ResNet)^[49]可以解决了深层网络训练困难的问题，如图 3.14 所示。随着网络的层数不断地加深，目标函数越来越容易陷入局部最优解。残差单元通过跳层连接的形式实现，即将单元的输入直接与单元的输出加在一起再激活。残差网络将拟合恒等映射函数转化为优化残差函数。当残差函数中时就构成了的恒等映射，这样就避免了多余网络层产生的冗余，使得网络在梯度下降的时候能够有效的应对网络退化问题。

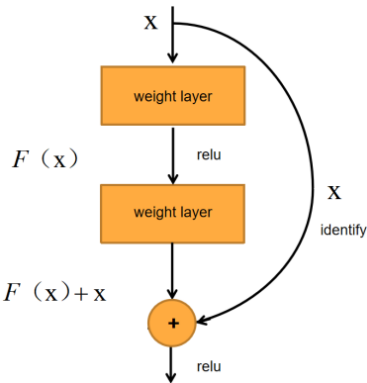


图 3.14 残差网络结构

通过引入 Focal Loss, 模型可以在训练过程中更加有效地应对样本分类不均衡的问题^[50]。它能够缓解由于样本数量不平衡所引起的模型偏向于多数类别的情况, 提高模型对于少数类别或难分类样本的识别能力。在处理二分类问题时, Focal Loss 的使用可以提升模型的性能, 并在不均衡数据集上取得更好的分类结果。

GRU 拥有记忆特性, 能够有效地处理具有时间序列特性的问题。但是在网络结构加深的情况下, 处理大量数据时的计算量依旧庞大。通过引入 CNN 进行特征提取, 能够对低维数据的深层次特征进行提取, 从而降低 GRU 部分训练所需的时间, 并提升模型的训练结果。当网络深度增加到一定程度时, 使用深度学习方法依旧面临着准确率降低的问题, 这是由于梯度消失现象导致的。在网络层数不断加深的同时, 输入层附近的非线性参数训练将变得困难。因此, 单纯的增加网络深度并不能直接提高模型效果。残差网络通过使用恒等映射的方法解决了网络深度加深时训练无法成功的问题。使用 Focal Loss 损失函数主要用于解决样本数量不均衡的问题。并使用 Early stopping 可以在模型训练整个过程中截取保存结果最优的参数模型, 以防止过拟合。最终构建的混合模型 ResCNN-GRU 模型结构如图 3.15 所示, 它使用了卷积神经网络 CNN 和门控循环单元 GRU 两种不同的神经网络结构, 并加入了残差结构作为连接, 以增强特征的传递和保留。ResCNN-GRU 定义了 CNN 卷积块, 它由三个卷积层组成, 每个卷积层后面跟随一个批量归一化层和一个 ReLU 激活函数层。第 1 个卷积层, 输入通道为 `chan_in`, 输出通道为 `chan_out`, 卷积核大小为 1, 第 2 个卷积层, 输入通道为 `chan_out`, 输出通道为 `chan_out`, 卷积核大小为 3, 第 3 个卷积层, 输入通道为 `chan_out`, 输出通道为 `chan_in`, 卷积核大小为 1。CNN 卷积块可以更好地提取输入向量中的特征, 以便后续模型更好地理解输入。ResCNN-GRU 通过一个由一维卷积层、批归一化层和 ReLU 激活函数构成的卷积块将输入长序列向量进行处理, 得到了一系列卷积特征。随后, 使用一系列由一维卷积层、批归一化层和 ReLU 激活函数构成的卷积块对这些特征进行进一步的处理。其中, 这些卷积块的设计使得网络可以在每一层对特征进行逐渐抽象和提炼, 从而获取更加高层次的特征信息。在经过一系列卷积块的处理后, 使用自适应平均池化层对特征进行降维, 从而获取更加全局的信息。随后, 将降维后的特征输入到 GRU 模型中进行处理, 以进一步提取时序信息, 可以捕捉时间序列的局部和全局特征, 提高模型的泛化性能。在 GRU 模型输出之后, 将其与卷积块的输出进行残差连接, 进一步提取特征信息。使用残差结构提高了模型的深度, 有助于提高模型的性能, 同时减轻了梯度消失的问题, 提高了训练效率。随后, 再次使用卷积块对特征进行处理, 最终将得到的特征输入到一系列卷积层和全连接层中, 进行分类预测。

ResCNN-GRU 的基础结构是由一系列的残差块组成的, 每个残差块由卷积块 CNN、门控循环单元和跳跃连接组成。卷积块 CNN 通过堆叠多层卷积、批归一化和激活函数的方式提取

局部特征，从而增强模型的表达能力。而 GRU 则是一种门控循环神经网络，它通过一系列的
门控机制控制信息的传递和遗忘，从而可以更好地捕捉序列数据的长依赖关系。在每个残差
块中，CNN 和 GRU 之间通过跳跃连接进行连接，使得局部特征和长依赖信息可以更好地融
合和传递。整个模型的输出通过全局平均池化层进行降维，将卷积层的输出转换为定长的向
量，然后通过全连接层进行分类。这种结构可以处理不同长度的输入序列，并提高模型的鲁
棒性。最终使用全连接层将特征映射到最终的分类结果上。具体的代码漏洞检测算法如算法
3.2 所示。

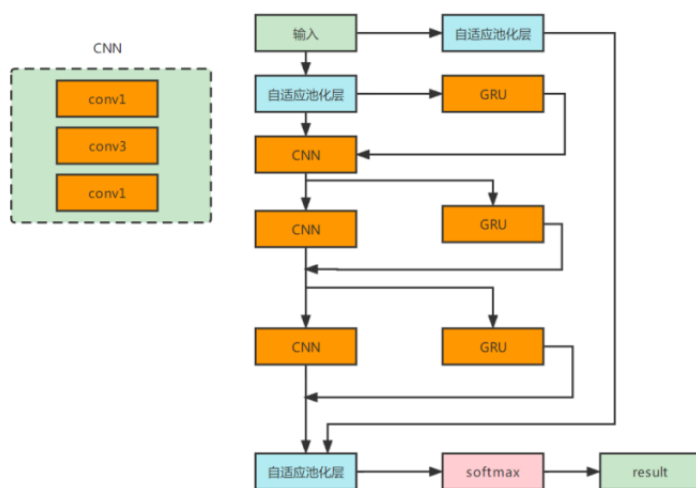


图 3.15 混合模型 ResCNN-GRU 模型

算法 3.2: 基于静态切片语义与度量代码的漏洞检测算法

输入: 程序 P

输出: 预测值 prediction # 0 表示不存在漏洞，1 表示存在漏洞

1. $llvm_ir \leftarrow compile_to_llvm_ir(P)$ #将程序 P 通过编译器 LLVM 的 Clang 转化为 LLVM IR 指令.
 2. $interest_points \leftarrow find_interest_points(llvm_ir)$ #调用的兴趣点选取算法,
 3. **For** interest_point in interest_points **do**
 4. #利用 SymPas 工具获取兴趣点的前向切片和后向切片
 5. $forward_slice \leftarrow SymPas.get_forward_slice(interest_point)$
 6. $backward_slice \leftarrow SymPas.get_backward_slice(interest_point)$
 7. #合并前向切片和后向切片
 8. $merged_slice \leftarrow merge_slices(forward_slice, backward_slice)$
 9. #使用 Inst2vec 将 LLVM IR 指令转化为向量表示
 10. $merged_slice_vector \leftarrow Inst2vec.convert_to_vector(merged_slice)$
 11. $merged_result \leftarrow merge_semantics_and_metrics(merged_slice_vector, metric_info)$
 12. **End for**
 13. #使用混合模型 ResCNN-GRU 进行训练
 14. $hybrid_model \leftarrow ResCNN_GRU()$
 15. $hybrid_model.train(training_merged_result)$
 16. $prediction \leftarrow hybrid_model.predict(test_merged_result)$
 17. **Return** prediction
-

3.6 本章小结

本章基于静态切片语义与度量代码的漏洞检测方法。首先,我们引入了缓冲区溢出漏洞、空指针解引用漏洞、SQL 注入攻击漏洞和函数调用漏洞这四种类型作为 LLVM IR 层面的兴趣点选取规则。为了检测这些兴趣点的缺陷,提出了基于 LLVM IR 的缺陷兴趣点检测算法。使用 SymPas 来获取兴趣点的前向切片和后向切片,并利用 Inst2vec 来获得 IR 指令的向量表示。为了进一步提高检测的准确性,结合了指令级切片度量。采用混合模型 ResCNN-GRU 进行训练,以得到最佳的检测结果。

本章的方法在研究源代码缺陷时专注于 LLVM IR 中间代码层面,具有良好的适应性和实用性。这意味着我们的方法不仅可以准确地检测出缺陷,而且可以适应不同的代码结构和编程语言。

第四章 基于动态切片与预训练模型的代码漏洞检测

上一章介绍了基于静态切片的代码漏洞检测方法，静态切片的计算成本较低，因为它不需要执行程序，而是通过静态分析技术直接分析源代码。这使得静态切片在大型代码库或复杂程序的分析中更具可行性和高效性，但有些漏洞只有在特定执行条件下才会被发现。本章提出了基于动态切片与预训练模型的代码漏洞检测方法。动态切片能够反映程序的实际情况，更加准确，能够发现一些只有在特定执行条件下才会出现的缺陷。使用 CodeBERT 预训练模型将程序切片语句块表示成二维张量，将其视为一通道灰度图像，将代码的结构和语义信息编码为像素值。借助计算机视觉模型 Swin Transformer 提取图像特征的能力，可以进一步提取图像中的关键漏洞特征，可以更准确地进行漏洞检测。

4.1 总体框架

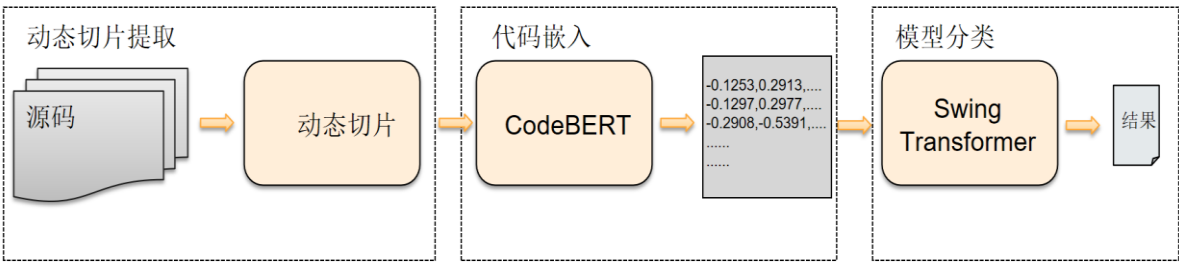


图 4.1 总体框架

该框架包括提取动态切片、代码嵌入和模型分类三个关键步骤。

在第一部分，通过获取源码的动态切片来获得更精确的变量切片。动态切片考虑了程序的执行路径和输入数据，相比于静态切片，它能够更真实地反映程序的实际情况，并发现一些只有在特定执行条件下才会出现的缺陷。通过动态切片，能够捕捉到程序的关键变量和相关语句，为后续的分析 and 处理提供了更准确的基础。

在第二部分，使用 CodeBERT 预训练模型将程序切片语句块表示为二维张量，将其视为一通道灰度图像。CodeBERT 模型是一种基于 Transformer 的预训练模型，具有良好的语义理解能力和跨语言通用性。通过将程序切片转化为二维张量表示，可以将代码的结构和语义特征编码为像素值，从而在向量空间中进行进一步的分析和处理。

在第三部分，使用 Swin Transformer 模型作为漏洞特征提取器和分类器。通过自注意力机制和局部感知机制，Swin Transformer 模型能够捕捉到程序切片中的重要特征，并提取有关漏洞的关键信息。这样能够将程序切片进行分类，识别出潜在的安全漏洞，并提供给开发人

员进行修复。

4.2 提取动态切片

动态切片（dynamic slicing）也是一种程序切片技术，具体来说，它基于程序的实际执行情况来提取程序切片，与静态切片不同，静态切片是在不考虑程序输入和输出情况的情况下，对程序进行分析，提取程序切片。当我们在程序中寻找特定错误或异常时，通常需要跟踪程序的执行路径，了解程序在哪些位置执行了哪些语句，相比于静态切片，动态切片得到的变量切片往往比静态切片得到的切片更精确。动态切片能够考虑程序的执行路径和输入数据，而静态切片只能考虑程序的静态结构。动态切片能够反映程序的实际执行情况，更加真实和准确，能够发现一些只有在特定执行条件下才会出现的缺陷。

动态切片可以根据输入数据的不同生成不同的切片结果，而静态切片只能生成一种结果。这使得动态切片能够更全面地覆盖程序的不同执行路径，更精准地发现缺陷。总之，动态切片在分析程序异常时可以提供更精确的信息，并且可以处理一些静态分析无法处理的动态程序特性，有助于提高程序分析的准确性和效率。

4.2.1 动态切片算法

动态切片准则：切片准则 $DSlice = (S, V, I)$ ， S 为程序中的某行语句， V 是 S 的变量子集， I 为程序输入。

```
i = 1; // S2
while (i <= n) { // S3
    x = read(); // S4
    if (x < 0) // S5
        y = f1(x); // S6
    else
        y = f2(x); // S7
    z = f3(y); // S8
    write(z); // S9
    i = i + 1; // S10
}
```

图 4.1 示例程序

在分析动态切片时往往会引入 execution history（执行历史）的概念。execution history 是一个可执行路径，它是由程序执行时所经过的节点组成。语句序列 $\langle S_1, S_2, \dots, S_n \rangle$ 表示在一个在给定输入下，程序依赖图中的顶点在执行期间被访问的顺序。在给定输入下，如果同一个语

句在 `execution history` 中出现多次, 那么用上标表示。 s_i^k 表示语句 s_i 的第 k 次执行。图 4.1 所示的程序, 当输入为 $n=2$, 两次输入的 x 分别为 -2 和 4 时, 程序的 `execution history` 为 $\langle 1, 2, 3, 4, 5, 6, 8, 9, 10, 3^2, 4^2, 5^2, 7, 8^2, 9^2 \rangle$ 。

典型的动态切片算法有四种, 分别是基于标记的动态切片、基于依赖性分析的动态切片、前向动态切片、基于动态依赖图的动态切片。基于动态依赖图的切片算法产生的切片结果比较精准。基于动态依赖图 DDG (Dynamic Dependence Graph) 的切片算法是由 Agrawal 和 Horgan 提出的^[38], 它通过建立动态依赖图来实现程序切片。

该算法的基本步骤如下:

- (1) 获取程序的执行记录。
- (2) 根据执行记录构建程序的动态依赖图。
- (3) 确定切片标准。
- (4) 遍历动态依赖图, 输出切片结果。

本文采用基于简化动态依赖图 (RDDG: Reduced Dynamic Dependence Graph) 的方法生成切片, 它是由 Agrawal 和 Horgan 对动态依赖图的进一步改进, 为了减少 DDG 中节点数。构造简化动态依赖图时需要两个表: DefnNode 表和 PredNode 表。最终的切片结果查找 ReachableNodes 来获得。

DefnNode 表: 变量名映射到上次赋值到这个变量中的依赖图节点。例如 $\text{DefnNode}[i]=n_2$ 表示变量 i 最后一次定义是在节点 n_2 中。

PredNode 表: 将控制语句 (if, while 等) 映射为该控制语句在执行历史中的最后一次出现相对应的节点。例如 $\text{PredNode}[S3]=n_3$, 表示控制语句在 `execution history` 中最后一次出现是在节点 n_3 中。

ReachableNodes: 节点依赖路径上的全部语句 (是语句不是节点, 不区分第几次执行)。

简化动态依赖图的生成算法步骤如下:

步骤一: 每当一个语句 S_i 被执行, 计算由最近一次对 S_i 引用的变量赋值的节点组成的集合 D , 以及由 S_i 控制依赖的节点组成的集合 C 。

步骤二: 对于 S_i 如果该语句之前已经执行过并且对应的结点是 n , 其直接后代与 $D \cup C$ 相同, 则 S_i 这次执行对应的节点依旧是 n ; 否则, 为 S_i 创建一个新的结点 n' , n' 所有的出边指向 $D \cup C$ 中的所有节点。

步骤三: 如果 S_i 对一些变量进行了赋值, 那么就将这些变量在 DefnNode 中的表项更新为 n' 。

步骤四: 如果 S_i 是一个条件判断, PredNode 中 S_i 对应的表项更新为 n' 。

为了防止在程序中存在环路依赖（由循环语句导致的），对于语句 S_i 在需要创建节点的时候（步骤二中）首先确定对应结点 n' 的直接后继结点集合 $D \cup C$ 中的任意一节点，记作 v （ n' 依赖于 v ），是否对 S_i 之前执行对应的结点产生依赖以及 n' 的其它直接后继结点是否从 v 可达，如果是则将 n' 与 v 合并。

构建了程序执行历史的简化动态依赖图之后，我们可以使用动态切片来获得任意变量 v 的相关语句集合。首先，我们需要在 **DefnNode** 表中找到与变量 v 相关的节点，然后去查询 **ReachableNodes** 集合，该集合中包含了源程序中与这些节点相关联的语句集合，这些语句构成了 v 的动态切片。

利用上述算法，对图 4.1 中的程序，在输入当输入为 $n=3$ ，两次输入的 x 分别为 -4 和 3 和 -2 时，生成的 **ReachableNodes** 如表 4.1 所示。

表 4.1 **ReachableNodes** 集合

节点	ReachableNodes 集合	节点	ReachableNodes 集合
1.	{1}	4 ² .	{1, 2, 3, 4, 10}
2.	{2}	5 ² .	{1, 2, 3, 4, 5, 10}
3.	{1, 2, 3}	7.	{1, 2, 3, 4, 5, 7, 10}
4.	{1, 2, 3, 4}	8 ² .	{1, 2, 3, 4, 5, 7, 8, 10}
5.	{1, 2, 3, 4, 5}	9 ² .	{1, 2, 3, 4, 5, 7, 8, 9, 10}
6.	{1, 2, 3, 4, 5, 6}	6 ² .	{1, 2, 3, 4, 5, 6, 10}
8.	{1, 2, 3, 4, 5, 6, 8}	8 ³ .	{1, 2, 3, 4, 5, 6, 8, 10}
9.	{1, 2, 3, 4, 5, 6, 8, 9}	9 ³ .	{1, 2, 3, 4, 5, 6, 8, 9, 10}
10,3 ² .	{1, 2, 3, 10}		

4.2.2 兴趣点的动态切片生成

根据第三章定义的兴趣点，我们目标是获取所有兴趣点在不同路径下的动态切片结果。通过借助符号执行工具 **KLEE**^[52]，符号执行可以用于生成测试用例以覆盖程序中的所有路径。符号执行将程序的输入变量符号化，然后以符号形式执行程序，生成路径约束。路径约束可以被求解器求解，得到满足路径约束的输入值，这些输入值即为覆盖该路径的测试用例。将借助符号执行得到的测试用例作为程序的输入，每一个输入都会对应一个 **execution history**，在获取到程序的执行记录后，根据执行记录构建程序的简化动态依赖图，由此便可获得执行路径上任意变量在该路径下的动态切片结果。

$$s_{val}^i = Slice[s, val, I_i] \quad (4.1)$$

$$S_{val} = \bigcup_{i=1}^n Slice[s, val, I_i] \tag{4.2}$$

$$S = \{S_{val1}, S_{val2}, ..., S_{valn}\} \tag{4.3}$$

s_{val}^i 表示兴趣点变量 val 在第 i 个执行路径下的动态切片结果， I_i 表示第 i 个测试用例， S_{val} 是该变量可能执行的所有路径的动态切片，是兴趣点变量 val 的最终动态切片结果。公式 4.3 是最终程序所有兴趣点变量的动态切片结果的集合。

```
#include <string.h>
int main() {
    char input[20];
    int flag;
    gets(input);
    scanf("%d", &flag);
    char buffer[10];
    if (flag == 1) {
        strcpy(buffer, input);
        printf("buffer: %s\n", buffer);
    }

    if(flag == 2) {
        strncpy(buffer, input, sizeof(buffer) - 1);
        buffer[sizeof(buffer) - 1] = '\0';
        printf("buffer: %s\n", buffer);
    }
    return 0;
}
```

图 4.2 缓冲区溢出示例

表 4.2 变量 buffer 切片结果表

	动态切片	前向后向静态切片
输入 1: flag=1;	{7,9,10,11}	{7,9,10,11,13,14,15,16}
输入 2: flag=2;	{7,13,14,15,16}	

当 flag 的值为 1 时，使用 strcpy 函数将输入字符串复制到 buffer 缓冲区中。strcpy 函数不会检查目标缓冲区的大小，如果源字符串 input 的长度超过了目标缓冲区 buffer 的大小，就会导致缓冲区溢出。当 flag 的值为 2 时，使用 strncpy 函数进行带有长度限制的复制操作。

“sizeof(buffer)-1”计算了目标缓冲区 buffer 的大小，并减去了 1，以确保在复制时留出一个

字节的空间用于字符串结束符\0。这样，可以避免在目标缓冲区中没有足够的空间容纳字符串结束符的情况下，避免了缓冲区溢出。

由此可见，flag 为 1 时可能存在缓冲区溢出；flag 为 2 或其他时，不存在缓冲区溢出。使用动态切片可以分别获得在 flag=1 和 flag=2 为程序输入时关于变量 buffer 的动态切片结果，相比于使用静态切片直接对变量 buffer 切片的得到的切片结果，动态切片结果更加精确，且能够体现该漏洞只有在特定执行条件下才会出现的缺陷。

4.3 代码嵌入

代码嵌入是将程序代码转化为连续向量表示的过程，使得计算机可以理解和处理代码的语义和结构。代码嵌入技术在代码分析、代码搜索、代码相似性计算、代码聚类等领域有着广泛的应用^[53]。本小节使用 CodeBERT 预训练模型进行代码嵌入，基于 CodeBERT 的代码嵌入具有以下优点，语义丰富性：CodeBERT 模型通过大规模的预训练能够理解代码的语义和上下文信息，将代码转化为语义丰富的向量表示，更好地捕捉了代码的含义和逻辑。跨语言通用性：CodeBERT 模型在预训练过程中使用了多语言代码语料库，因此可以处理不同编程语言的代码，并产生一致的嵌入表示。上下文敏感性：CodeBERT 考虑了代码的上下文信息，将代码语句块作为整体进行编码，而不是独立地对每个代码语句进行编码。这样可以更好地捕捉代码之间的依赖关系和语义关联。通过基于 CodeBERT 的代码嵌入技术，我们可以将代码转化为语义丰富的向量表示，从而在代码分析、代码搜索、代码相似性计算等任务中获得更好的效果和性能。

4.3.1 CodeBERT 预训练模型

CodeBERT 是一种自然语言文本和编程语言代码预训练模型，其模型架构包括 12 个层，每个层具有 12 个自注意力头，每个头的大小为 64。隐藏维度为 768，前馈层的内部隐藏层大小为 3072，模型参数总量为 1.25 亿。在预训练阶段，研究者将输入设置为两个片段和一个特殊分隔符的组合，其中一个片段是自然语言文本，另一个是以编程语言写成的代码。CodeBERT 的输出包括每个 token 的语境向量表示以及[CLS]的表示，用于聚合序列表示。CodeBERT 在大规模数据集上进行了训练，经过参数微调后，可以用于新任务的数据集，并在新任务上达到最优性能^[54]。CodeBERT 使用了一种名为掩码语言建模（MLM）的方法，使用一个包含 NL 单词序列（w）和 PL 标记序列（c）的数据点作为输入。该方法会随机选定一组位置（mw 和

mc) 用[MASK]标记替换选定的位置, 以屏蔽掉 15% 的标记。

$$\begin{aligned}
 m_i^w &\sim \text{unif}\{1, |\mathbf{w}|\} \text{ for } i = 1 \text{ to } |\mathbf{w}| \\
 m_i^c &\sim \text{unif}\{1, |\mathbf{c}|\} \text{ for } i = 1 \text{ to } |\mathbf{c}| \\
 \mathbf{w}^{\text{masked}} &= \text{REPLACE}(\mathbf{w}, \mathbf{m}^w, [\text{MASK}]) \\
 \mathbf{c}^{\text{masked}} &= \text{REPLACE}(\mathbf{c}, \mathbf{m}^c, [\text{MASK}]) \\
 \mathbf{x} &= \mathbf{w} + \mathbf{c}
 \end{aligned} \tag{4.4}$$

MLM 目标是预测被遮盖掉的原始标记, 其公式如下所示, 其中 p^{D_1} 是鉴别器, 可以从一个大词汇表中预测一个标记。

$$\mathcal{L}_{\text{MLM}}(\theta) = \sum_{i \in \mathbf{m}^w \cup \mathbf{m}^c} -\log p^{D_1}(x_i | \mathbf{w}^{\text{masked}}, \mathbf{c}^{\text{masked}}) \tag{4.5}$$

RTD 是替换标记检测的缩写, 它是用于有效学习自然语言预训练模型的一种目标。在 MLM 目标中, 只使用双峰数据 (即 NL-PL 对的数据点) 进行训练, 而在这里则利用了双峰和单峰数据进行训练。为了实现这个目标, 使用了两个数据生成器: NL 生成器 p^{G_w} 和 PL 生成器 p^{G_c} , 它们都会为一组随机屏蔽的位置生成可信的替代方案。

$$\begin{aligned}
 w_i &\sim p^{G_w}(w_i | \mathbf{w}^{\text{masked}}) \text{ for } i \in \mathbf{m}^w \\
 \hat{c}_i &\sim p^{G_c}(c_i | \mathbf{c}^{\text{masked}}) \text{ for } i \in \mathbf{m}^c \\
 \mathbf{w}^{\text{corrupt}} &= \text{REPLACE}(\mathbf{w}, \mathbf{m}^w, \mathbf{w}) \\
 \mathbf{c}^{\text{corrupt}} &= \text{REPLACE}(\mathbf{c}, \mathbf{m}^c, \hat{\mathbf{c}}) \\
 \mathbf{x}^{\text{corrupt}} &= \mathbf{w}^{\text{corrupt}} + \mathbf{c}^{\text{corrupt}}
 \end{aligned} \tag{4.6}$$

判别器的训练目标是二元分类问题, 即判断一个词是否为原始词。这意味着判别器被训练用于区分真实的词和生成的词, 并将它们分为两类: 原始的和非原始的。这个目标类似于生成对抗网络中的判别器, 但与 GAN 不同, 如果生成器恰好产生了正确的标记, 则该标记的标签为“真实”而不是“伪造”。对于判别器的损失函数, 使用了指示器函数 $\delta(i)$ 和预测第 i 个词为原始词的概率 p^{D_2} 。指示器函数 $\delta(i)$ 的作用是将判别器预测的标签和实际标签进行比较, 以计算误差。概率 p^{D_2} 是判别器的输出, 表示第 i 个词为原始词的概率。通过最小化损失函数, 可以使判别器更准确地区分真实词和生成词。

$$\begin{aligned}
 \mathcal{L}_{\text{RTD}}(\theta) &= \sum_{i=1}^{|\mathbf{w}|+|\mathbf{c}|} \left(\delta(i) \log p^{D_2}(\mathbf{x}^{\text{corrupt}}, i) + (1 - \delta(i)) (1 - \log p^{D_2}(\mathbf{x}^{\text{corrupt}}, i)) \right) \\
 \delta(i) &= \begin{cases} 1, & \text{if } x_i^{\text{corrupt}} = x_i \\ 0, & \text{otherwise} \end{cases}
 \end{aligned} \tag{4.7}$$

两种生成器的实现方法，采用的是 n -gram 语言模型，而且采用了双向上下文的设计。一种用于处理 NL，一种用于处理 PL。NL 和 PL 是不同的数据模态，因此需要分别从它们的单模数据点中学习相应的语言模型。这种方法还可以轻松地推广到学习双模生成器或使用更复杂的生成器，例如基于 Transformer 的神经结构。最终训练的损失函数由 MLM 和 RTD 的损失函数构成。

$$\min_{\theta} \mathcal{L}_{\text{MLM}}(\theta) + \mathcal{L}_{\text{RTD}}(\theta) \quad (4.8)$$

在预训练阶段，我们将输入设置为两个片段的连接，中间使用一个特殊的分隔符标记 [SEP]，如图 4.3 所示，具体来说，输入序列的格式为 [CLS], $w_1, w_2, \dots, w_n$, [SEP], $c_1, c_2, \dots, c_m$, [EOS]。其中，一个片段是自然语言文本，另一个片段是来自某种编程语言的代码。[CLS] 是在两个片段前面的一个特殊标记，其最终的隐藏表示被认为是用于分类或排序的聚合序列表示。

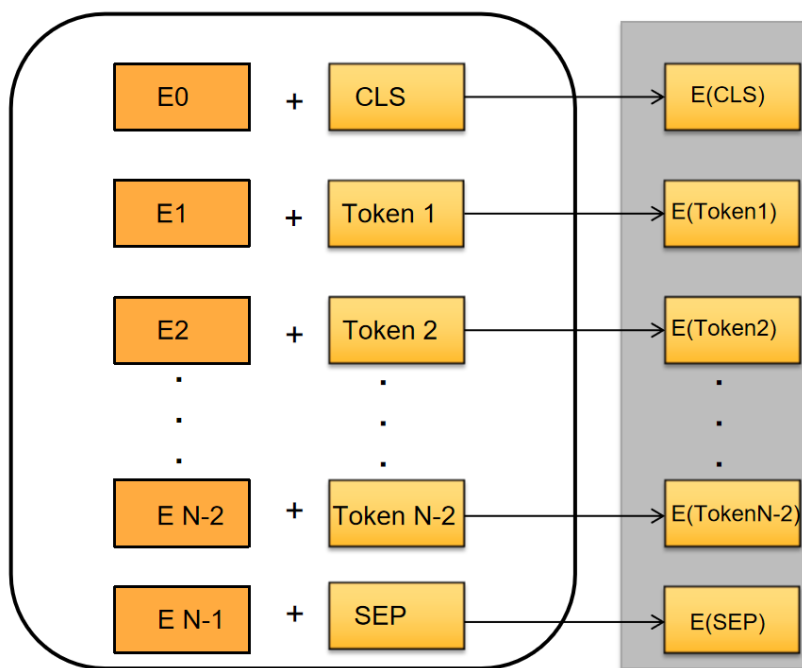


图 4.3 CodeBERT 的部分结构

许多嵌入任务通常需要检查连续信息才能生成语义丰富的向量表示，易受攻击的模式通常包含声明、赋值、控制流和其他操作逻辑。因此，需要关注易受攻击函数中的多个关键点。多头注意力使算法能够关注多个关键点，从而有助于捕捉潜在易受攻击的程序模式。此外，易受攻击模式是长期依赖的，这意味着长距离的上下文信息对于易受攻击函数的检测至关重要。对于易受攻击函数，如单词 x_i 的 x_{i-n} 和 x_{i+n} 也很有用。为了获取单词周围单词的依赖关系，位置编码被设计出来。位置编码层被添加到输入嵌入层中，位置编码的维度与嵌入的维

度相同,因此两个参数可以相加。由于源代码结构遵循逻辑规则或正式参数,并与语言或逻辑中的含义相关联,因此它们是紧密连接和紧密耦合的。因此,容易受攻击的代码片段通常与先前或后续代码链接或连接的组成部分有关,甚至可能与两者都有关。一个小的易受攻击代码片段通常包含多行代码,这些代码可以分布在函数块中。对于传统的嵌入模型,如 Word2Vec,需要花费大量的工作或技巧才能确定哪行代码导致了软件漏洞;但是,对于 Transformer 的结构,确认每个单词之间的距离几乎没有困难。因此,CodeBERT 的结构可以使模型更轻松地检测前向和后向的长期依赖性,从而有效捕捉易受攻击的代码漏洞。

通过对兴趣点进行程序切片获得相应的代码语句块 $[C_1, C_2, C_3 \dots C_k]$,将代码语句块进行了 token 化和位置嵌入处理,以便输入到深度学习模型中进行训练和推理。考虑到 CodeBERT 输入 token 数量是有 512 的上限,因此本文将先每一行转化为 token 序列,如 C_1 这行代码含有 n_1 个 token,通过 CodeBERT 进行代码嵌入之后,将生成 $(n_1+2)*768$ 维度的语义向量,最后将语句块中每行生成的语义向量拼接为一个整体,变成 $(n_1+n_2+\dots+n_k)*768$ 维度的语义向量。

例如,首先需要将这行代码 `int a = 3` 作为输入转化为 token 序列,其中包括特殊 token [CLS]、`int`、`a`、`=`、`3`、[SEP]。[CLS] 是用作输入序列首端的特殊 token,其隐藏表示可以用于分类或排序的聚合序列表示。[SEP] 则是作为输入序列的结尾标记。然后将 token 序列与其对应的位置信息进行嵌入。这里使用了与 BERT 模型相同的位置嵌入方式,即将位置信息添加到编码层最底部的嵌入过程中。位置嵌入与令牌嵌入具有相同的维度,因此可以对两者进行叠加。具体公式如下:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$
(4.9)

CodeBERT 在大规模数据集上训练出一个较好的模型,可以用在新任务的数据集,通过参数微调后的 CodeBERT 模型可在新任务上达到最优性能。Word2Vec 为语料库中的每个单词生成一个唯一的向量;但是,它忽略了单词之间的联系,CodeBERT 可以根据上下文信息生成不同的向量。如图 4.4,图上部分的 add 函数存在整形溢出的错误,图下部分的 add 函数加入了 if 判断条件,可以避免整形溢出的这个缺陷,图中右半部分是 CodeBERT 生成的不同的语义信息。

<pre>int add (int a){ int b = 2147483647; b = b+ a; return b; }</pre>	-0.1253, 0.2913, -0.0365, ..., -0.4400, -0.3012, 0.3963 -0.1297, 0.2977, -0.0316, ..., -0.4477, -0.3031, 0.4033 -0.2908, -0.5391, 0.0063, ..., 0.2980, -0.8603, 1.0207 -0.1294, 0.2975, -0.0317, ..., -0.4475, -0.3030, 0.4030
<pre>int add (int a){ int b = 2147483647; If(a<0){ b = b+ a; } return b; }</pre>	-0.1129, 0.2888, -0.0732, ..., -0.3652, -0.3289, 0.3890 -0.1191, 0.2957, -0.0679, ..., -0.3734, -0.3321, 0.3952 -0.1995, -0.6103, 0.0399, ..., 0.2919, -0.7986, 0.9302 -0.1188, 0.2956, -0.0680, ..., -0.3733, -0.3320, 0.3950

图 4.4 add 函数示例

使用不同的序列长度对漏洞检测结果会产生不同的影响。在将代码语句块映射为成 $n*768$ 维度的语义向量，如果代码过长需要截断过长的代码，以将代码转换为固定长度的向量，以平衡过长的向量和信息丢失之间的关系。如果函数的代码序列不够长，则会用 0 填充。序列长度显著影响 CodeBERT 模型的性能，当输入序列太长时，需要截断，导致信息丢失，模型无法完全学习切片语句块的完整的特征。当序列过短时，需要将以 0 填充在末尾，无用信息将占据大部分输入序列的空间。此外，程序中还存在着远距离依赖性，因为一些易受攻击的语句可能与其关注的兴趣点相距很远。由于 CodeBERT 的双向结构，在距离太远时，前面的信息可能从模型的记忆中消失。因此，需要适当的输入序列长度，并使模型专注于易受攻击性的特征。

4.3.2 基于 CodeBERT 和程序切片的代码表示算法

基于 CodeBERT 和程序切片的代码表示算法如算法 4.1 所示，用于将给定程序切片语句块 C 表示为一个 $L*768$ 维度的二维张量，其中 L 是合适的固定长度，768 是 CodeBERT 预训练模型生成的向量维度。该算法首先对语句块 C 中的每一行代码进行处理，用 CodeBERT 预训练模型将每行代码的 k_i 个 token 转换为一个 $(k_i+2)*768$ 维度的矩阵。然后将每行生成的 $(k_i+2)*768$ 维度的向量拼接成一个 $K*768$ 维度的矩阵。如果矩阵 K 的长度小于合适长度 L，则在末尾用 0 补齐。如果矩阵 K 中兴趣点所在的 token 的上边部分的长度小于 $L/2$ ，则截取下边部分，使得整个矩阵变成 $L*768$ 维度。如果矩阵 K 中兴趣点所在的 token 的下边部分的长度小于 $L/2$ ，则截取上边部分，使得整个矩阵变成 $L*768$ 维度。如果矩阵 K 中

兴趣点所在的 token 上下两部分的长度都大于等于 $L/2$ ，则保留兴趣点上边 $(L-1)/2$ 部分和下边 $(L-1)/2$ 部分，使得整个矩阵变成 $L * 768$ 维度。最终输出 $L * 768$ 维度的二维张量，表示程序切片语句块 C 的 CodeBERT 表示。

算法 4.1: 基于 CodeBERT 和程序切片的代码表示算法

输入: 程序切片语句块 $C=\{C_1, C_2, C_3 \dots C_k\}$

输出: 语句块 C 的 $L * 768$ 的二维张量

```

1.  $V \leftarrow []$  # 初始化空的向量列表 V
2. For  $C_i$  in C do
3.   # 使用 CodeBERT 预训练将  $C_i$  中的  $k_i$  个 token 生成  $(k_i+2)*768$  维度的张量
4.    $vector\_ci \leftarrow codebert\_embedding(ci)$ 
5.    $V.append(vector\_ci)$ 
6.   # 将每行生成的  $(k_i+2)*768$  维度的向量拼接为一个二维张量，形状为  $K*768$ 
7.    $K \leftarrow concatenate\_vectors(V)$ 
8. End for
9. If  $K.shape[0] < L$  then # 判断 K 的长度是否小于合适的长度 L
10.    $padded\_K \leftarrow pad\_tensor(K, L)$  # 在末尾用 0 补齐，使整体张量形状为  $L*768$ 
11.   return  $padded\_K$ 
12. End if
13. Else
14.   # 计算兴趣点所在 token 的上边部分和下边部分的长度
15.    $upper\_length \leftarrow calculate\_upper\_length(interesting\_point\_token)$ 
16.    $lower\_length \leftarrow calculate\_lower\_length(interesting\_point\_token)$ 
17.   # 根据不同情况截断或保留部分张量，使整体形状变为  $L*768$ 
18.   If  $upper\_length < L/2$  then
19.      $truncated\_K \leftarrow truncate\_lower\_part(K, L)$ 
20.   End if
21.   Else if  $lower\_length < L/2$  then
22.      $truncated\_K \leftarrow truncate\_upper\_part(K, L)$ 
23.   End if
24.   Else  $truncated\_K \leftarrow truncate\_both\_parts(K, L)$ 
25. Return  $truncated\_K$ 

```

4.4 基于 Swin Transformer 模型的漏洞分类

Transformer 的网络结构主要由以下几个部分组成:输入嵌入层(Input Embedding Layer):将输入的图像划分成若干个小块,然后对每个小块进行卷积操作得到特征图,最后将这些特征图通过线性变换映射到一个向量空间中。Swin Block:这是 Swin Transformer 的核心模块,它包括多层 Transformer 编码器,每个编码器包括多头注意力机制和前馈神经网络。交叉连接层(Cross-Stage Connection Layer):在 Swin Block 内部,采用了跨阶段连接技术,即每个 Swin Block 的输出会与下一个 Swin Block 的输入相连接,从而实现信息的传递和共享。局部窗口注意力机制(Local Window Attention):由于 Swin Transformer 在计算机视觉任务中应用,因

此采用了局部窗口注意力机制来减少计算量和内存占用。这种局部窗口注意力机制可以被看作是一种可分离的卷积操作，可以在处理图像时实现空间上的位置感知。输出层（Output Layer）：将 Swin Transformer 的最后一个 Swin Block 的输出通过线性变换映射到相应的类别，得到最终的分类结果。Swin Transformer 是一种新型的视觉模型，是在原有的 Transformer 模型基础上提出的一种改进。与传统的 Transformer 模型不同的是，Swin Transformer 使用了分层结构和窗口位移机制，以此来解决传统 Transformer 模型在处理大尺度图像时的效率问题。本节使用 CodeBERT 预训练模型将切片语句块转成 $K \times 768$ 的二维张量，可以将其看作 $K \times 768 \times 1$ 的一通道图片，通过 Swin Transformer 来进行漏洞分类。Swin Transformer 的网络结构如图 4.5 所示。

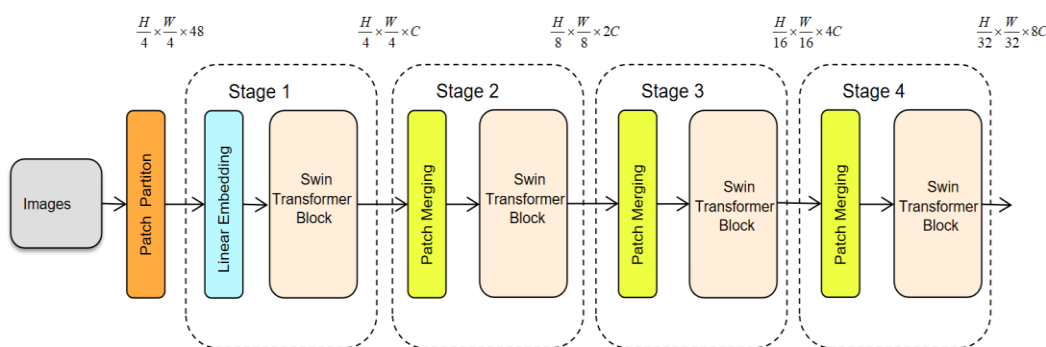


图 4.5 Swin Transformer 的网络结构

Swin Transformer Block 通过将标准的多头自注意力 (MSA) 模块替换为基于位移窗口的模块来构建，同时保持其他层不变。标准 Transformer 架构利用全局自注意力，其中计算一个 token 与所有其他 token 之间的关联。这个过程往往对于涉及高分辨率图像的许多视觉任务来说，与 token 数量的平方成比例的计算复杂度是极其不可行的。为了解决这个问题，自注意力是在局部窗口内计算的。这些窗口以非重叠的方式划分图像。假设每个窗口有 $M \times M$ 个 patches，则全局多头自注意力模块和一个基于窗口的模块在大小为 $h \times w$ 的 patch 图像上的计算复杂度为：

$$\begin{aligned}\Omega(\text{MSA}) &= 4hwC^2 + 2(hw)^2C \\ \Omega(\text{W-MSA}) &= 4hwC^2 + 2M^2hwC\end{aligned}\quad (4.10)$$

全局的多头自注意力模块的时间复杂度随着 patch 数量的增加呈二次复杂度增长，这使得它在高分辨率图像中应用时不可行。相比之下，基于窗口的方法在 patch 数量上呈现线性时间复杂度，只要窗口大小 M 固定不变。但是，基于窗口的自注意力技术中窗口之间缺乏连接，限制了其功能和灵活性。为了进一步建模跨窗口连接，提出了一种 Shifted Window Partitioning

方法。第一个模块使用常规的从左上角像素开始的窗口分割方案。然后，下一个模块使用窗口设置，其与前一层正常分割窗口相比偏移了 $\frac{M}{2} \times \frac{M}{2}$ 个像素。连续的 Swin Transformer Blocks 的计算如下：

$$\begin{aligned}\hat{z}^l &= W - \text{MSA}\left(\text{LN}\left(z^{l-1}\right)\right) + z^{l-1}, \\ z^l &= \text{MLP}\left(\text{LN}\left(\hat{z}^l\right)\right) + \hat{z}^l, \\ \hat{z}^{l+1} &= SW - \text{MSA}\left(\text{LN}\left(z^l\right)\right) + z^l, \\ z^{l+1} &= \text{MLP}\left(\text{LN}\left(\hat{z}^{l+1}\right)\right) + \hat{z}^{l+1},\end{aligned}\tag{4.11}$$

Swin Transformer 将输入的图像按照非重叠的方式划分为一系列大小为 4×4 的 patch，每个 patch 作为一个 token，由于每个小块的大小是 $4 \times 4 \times 3$ ，因此每个 token 的特征维度为 48。接着，将这些 token 送入一个线性嵌入层中，并将其嵌入到一个 C 维的向量空间中。然后将这些向量输入到一系列的 Swin Transformer Block 中进行特征转换。这些模块保持 token 的数量不变（即 $\frac{H}{4} \times \frac{W}{4}$ ）。随着网络层数的增加，每个模块的输出会经过一个 patch merging 层，以减少 token 的数量。第一个 patch merging 层将相邻的 2×2 个小块的特征连接在一起，并应用一个线性层将连接的特征变换为 2C 维。之后，再次使用 Swin Transformer 模块进行特征转换，保持分辨率为 $\frac{H}{8} \times \frac{W}{8}$ 。这个过程再次重复两次，分别得到分辨率为 $\frac{H}{16} \times \frac{W}{16}$ 和 $\frac{H}{32} \times \frac{W}{32}$ 的特征图。例如输入的 RGB 三通道图片为 $224 \times 224 \times 3$ ，输入的图片为 $224 \times 224 \times 3$ 经过 Patch Partition 展平变成 $56 \times 56 \times 48$ 。在第一阶段，经过 Linear Embedding 层，将每个 48 维的特征向量映射为 96 维，得到大小为 $56 \times 56 \times 96$ 的特征图。然后经过 Swin Transformer Block，特征图大小不变，仍为 $56 \times 56 \times 96$ 。在第二阶段，经过第一个 Patch Merging 层，将相邻的 4 个 patches 合并成一个 2×2 的 patch，然后再通过一个 Linear 层将特征向量的维度从 96 扩展到 192，得到大小为 $28 \times 28 \times 192$ 的特征图。经过 Swin Transformer Block，特征图大小不变，仍为 $28 \times 28 \times 192$ 。在第三阶段，经过第二个 Patch Merging 层，将相邻的 4 个 2×2 的 patches 合并成一个 4×4 的 patch，然后再通过一个 Linear 层将特征向量的维度从 192 扩展到 384，得到大小为 $14 \times 14 \times 384$ 的特征图。经过 Swin Transformer Block，特征图大小不变，仍为 $14 \times 14 \times 384$ 。在第四阶段，经过第三个 Patch Merging 层，将相邻的 4 个 4×4 的 patches 合并成一个 8×8 的 patch，然后再通过一个 Linear 层将特征向量的维度从 384 扩展到 768，得到大小为 $7 \times 7 \times 768$ 的特征图。经过 Swin Transformer Block，特征图大小不变，仍为 $7 \times 7 \times 768$ 。最终，将得到大小为 $7 \times 7 \times 768$ 的特征图，该特征图被送入一个全局平均池化层中，然后经过一个全连接层用于分类。基于 Swin Transformer 的代码漏洞检测算法如算法 4.2 所示。

算法 4.2: 基于 Swin Transformer 的代码漏洞检测算法

输入: 程序 P

输出: classification_result# 0 表示不存在漏洞, 1 表示存在漏洞

```
1.  dynamic_slices ← [] # 初始化列表, 存储不同兴趣点的动态切片结果
2.  For interest_point in interest_points do # 兴趣点选取并进行动态切片
3.      dynamic_slice ← getDynamicSlice(p, execution_path, input_data)
4.      dynamic_slices.append(dynamic_slice)
5.  End for
6.  #使用 CodeBERT 模型进行表示学习
7.  tensor_representation ← codeBERTRepresentation(dynamic_slice)
8.  #使用 Swin Transformer 模型提取漏洞特征和分类
9.  features ← SwinTransformer_extract_features(tensor_representation)
10. classification_result ← SwinTransformer_classify(features)
11. Return classification_result
```

4.5 本章小结

本章介绍了基于动态切片与预训练模型的代码漏洞检测方法。该方法利用动态切片技术反映程序的实际执行情况,能够准确发现在特定执行条件下出现的缺陷。通过使用 CodeBERT 预训练模型将程序切片表示为二维张量,并将其视为灰度图像来编码代码的结构和语义信息,从而更全面地捕捉代码的语义特征。同时,借助 Swin Transformer 提取特征的能力,可以进一步提取图像中的关键漏洞特征,实现更准确的漏洞检测,通过实验验证,动态切片与预训练模型的代码缺陷方法,能适应多种的漏洞类型,此外预训练模型通过在大规模数据上进行训练,学习到了丰富的语义和结构特征,这使得模型能够更好地理解代码的语义信息。

第五章 实验结果与分析

本章对第三章的基于静态切片语义与度量代码的漏洞检测和第四章的基于动态切片与预训练模型的代码缺陷检测分别进行了实验，并对实验结果进行了分析。

5.1 基于静态切片语义与度量代码的漏洞检测的实验与分析

5.1.1 实验数据集选取

数据集选取实验数据集来自于美国国家标准与技术研究院 (National Institute of Standards and Technology, NIST) 的软件保证参考数据集 (Software Assurance Reference Dataset, SARD)^[51], 本章中涉及到的缺陷类型包含 CWE134 (使用外部控制的格式字符串)、CWE121 (堆栈缓冲区溢出)、CWE119 (内存缓冲区)、CWE-399 (资源管理漏洞) 等。这些数据集中的代码分为了两类: 存在缺陷代码为 bad, 不存在缺陷代码为 good。通过对数据集中的代码通过程序切片有 83762 个, 其中和指针相关的错误有 21358 个, 数组相关的错误有 12834, API 调用相关错误的有 41983 个, 算数表达式相关错误有 7587 个, 训练集、验证集、测试集比为 7:2:1。

5.1.2 评估指标

评估模型性能的指标包括准确率、误报率、漏报率、查准率和 F1 分数。准确率 (Accuracy, 简称 Acc) 准确率指的是在所有样本中, 正确分类的样本数占总样本数的比例。其中 TP 表示真正例 (检测出的漏洞), TN 表示真负例 (没有检测出的漏洞), FP 表示假正例 (错误地将非漏洞样本分类为漏洞), FN 表示假负例 (没有检测出的漏洞)。准确率越高, 说明模型的准确性越高。误报率 (False Positive Rate, 简称 FPR) 误报率指的是所有负样本中被错误分类为正样本的样本数占有所有负样本的比例。误报率越低, 说明模型误报的概率越小, 即把非漏洞样本分类为漏洞样本的错误率越低。漏报率 (False Negative Rate, 简称 FNR) 漏报率指的是所有真正样本中未被检测出来的漏洞样本数占总正样本数的比例。漏报率越低, 说明模型检测出漏洞的能力越强。查准率 (Precision, 简称 P) 查准率指的是所有被检测出来的正样本中, 真正样本的比例。查准率越高, 说明模型检测出漏洞的准确性越高。F1 分数 (F1-Score) 是查准率和查全率的加权平均值, 反映了模型的综合性能。F1 分数越高, 说明模型的综合性能

能越好。具体公式如下:

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \tag{5.1}$$

$$FPR = \frac{FP}{FP + TN} \tag{5.2}$$

$$FNR = \frac{FN}{TP + FN} \tag{5.3}$$

$$P = \frac{TP}{TP + FP} \tag{5.4}$$

$$F1 = \frac{2 \times P \times TPR}{P + TPR} \tag{5.5}$$

5.1.3 实验环境及参数设置

表 5.1 实验环境表

环境	配置
操作系统	Windows 10
开发语言	Python
Python 版本	Python3.7
Torch 版本	1.11.0
Torchvision 版本	0.12.0
内存	16GB
硬盘	512GB

表 5.2 实验参数表

参数名称	参数值
优化器 Adam	0.001
损失函数	focal loss
metrics	accuracy
Early_stoping epoch	12
Epoch	50
batch_size	64
Dropout	0.5

5.1.4 实验结果分析

本章所设计的实验用于回答以下 3 个研究问题。

RQ1: 本章方法采用 LLVM IR 中间语言来表示语义信息和度量, 相对于源码表示, 能否更好的检测代码的缺陷?

RQ2: 本章方法提出的切片度量, 能否更好地体现代码的缺陷特征?

RQ3: 本章方法相较于其他漏洞检测系统, 是否有更好的漏洞检测性能?

针对 RQ1 的实验, 从源码和 LLVM IR 中间语言两个角度进行实验, One-Hot 编码、Word2vec 是从源码层面进行代码嵌入, 而 Inst2vec 是从 LLVM IR 中间语言层面进行代码嵌入。首先使用 Inst2vec 对指令进行向量表示, 分别选取 64、128、256、1024 的向量长度, 实验结果如图 5.1 所示, 在向量长度为 512 时, 准确率最高, 语义与度量相结合之后的准确率达到 94.1%, 比只有语义信息高 1.4%。并分别对比了源码层面的 One-Hot 编码、Word2vec 两种方法, 实验结果如表 5.3 所示, 为了更好地将代码的语义信息保留在向量中, 本章通过对 One-Hot 编码、Word2vec、Inst2vec 方法做了对比实验, 通过最终的结果指标来衡量它们的优劣。Inst2vec 的结果要明显好于 One-Hot 编码、Word2vec。One-Hot 编码没有考虑代码上下文的关系, Word2vec 利用神经网络对词的上下文训练得到词的向量化表示, 考虑代码的上下文, 与 Word2vec 方法相比, Inst2vec 效果要更好, 准确率提升了 4.5%, 精度提升了 3.3%。在 LLVM-IR 中间语言层面的方法不仅可以扩展到其他源码语言, 而且可以更好地检测代码的缺陷。

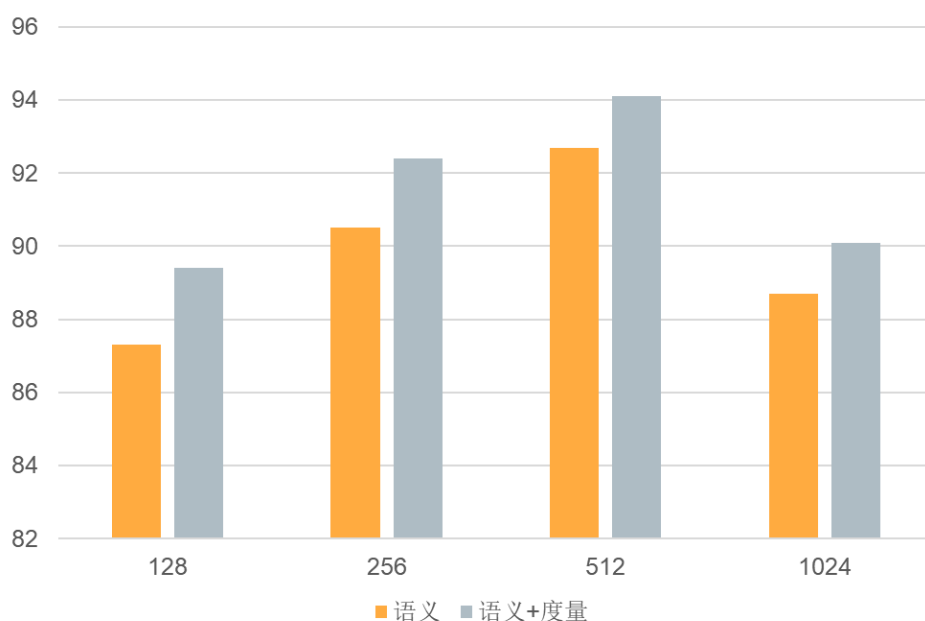


图 5.1 不同长度的对比实验图

表 5.3 不同代码嵌入方法下各指标对比结果

Method	Acc(%)	P(%)	F1(%)	FPR(%)	FNR(%)
One-Hot	76.4	77.2	72.9	8.9	30.1
Word2vec	89.6	89.2	86.9	6.8	15.6
Inst2vec	94.1	92.5	93.8	4.5	4.9

针对 RQ2 的实验，针对切片度量能否对预测代码缺陷，经过四组对比实验，本文选择了四种不同长度的向量作为模型输入，并将切片语义特征和度量特征共同加入到模型中，得到了表 5.4 中的结果。通过实验发现，加入度量特征后，预测缺陷的准确率可以提高 1.3%-2.4%，表明度量特征对于缺陷预测具有良好的效果。此外，本章还对比了使用 LSTM、GRU 和本文方法进行对比，分别语义和语义融合度量两个方面进行实验。在 LSTM 网络中，语义和切片度量融合的实验结果表明，在准确率、精度、F1 分数三个指标上分别提升了 1.6%、4.0%、3.9%，在误报率、漏报率上分别下降了 2.9%、3.6%。在 GRU 网络中，语义和切片度量融合的实验结果表明，在准确率、F1 分数两个指标上分别提升了 1.7%、2.7%，在精度、误报率、漏报率上分别下降了 2.5%、2.2%、4.2%。在本章方法网络中，语义和切片度量融合的实验结果表明，在准确率、精度、F1 分数三个指标上分别提升了 1.6%、1.4%、4.0%，在误报率、漏报率上分别下降了 1.5%、6.5%。可以发现，在不同的神经网络模型中，加入度量特征可以有效地提高模型预测缺陷的准确率。

表 5.4 不同神经网络下各指标对比结果

Method	Acc(%)	P(%)	F1(%)	FPR(%)	FNR(%)
LSTM（语义）	87.5	85.6	81.8	11.8	21.4
LSTM （语义+度量）	89.1	89.6	85.7	8.9	17.8
GRU （语义）	90.4	93.7	90.1	8.3	13.4
GRU （语义+度量）	92.1	91.2	92.8	6.1	9.2
本文方法 （语义）	92.5	91.1	89.8	6.0	11.4
本文方法 （语义+度量）	94.1	92.5	93.8	4.5	4.9

针对 RQ3 的实验，如表 5.5 所示，实验结果表明：本章提出的方法在准确率、精度、F1

值、误报率和漏报率等指标上都表现优异。相较于 VulDeePecker 和 SySeVR，本文方法的准确率分别提高了 5.0%和 1.6%，精度分别提高了 2.9%和 1.4%，F1 值分别提高了 12.0%和 4.0%，误报率分别降低了 4.4%和 1.5%，漏报率分别降低了 12.9%和 6.6%。这些改进得益于本文方法对模型的改进以及采用 Focal Loss 损失函数来解决正负样本不均衡问题。此外，本文提取与漏洞相关的语义信息时，从 LLVM IR 的中间表示找到缺陷兴趣点，并进行切片，可以更好地体现代码的缺陷。另外，本文方法还融合了切片度量，对于预测代码缺陷也有很好的效果，这也是本文在误报率和漏报率这两个指标上表现优异的原因之一。

表 5.5 不同代码漏洞检测系统下各指标对比结果

Method	Acc(%)	P(%)	F1(%)	FPR(%)	FNR(%)
Flawfinder	69.8	60.1	59.8	26.6	40.5
Checkmarx	74.4	72.8	69.3	8.2	33.8
VulDeePecker	89.1	89.6	81.8	8.9	17.8
SySeVR	92.5	91.1	89.8	6.0	11.5
本章方法	94.1	92.5	93.8	4.5	4.9

5.2 基于动态切片与预训练模型的代码漏洞检测的实验与分析

5.2.1 实验数据集选取

数据集选取实验数据集来自于美国国家标准与技术研究院（National Institute of Standards and Technology, NIST）的软件保证参考数据集（Software Assurance Reference Dataset, SARD）^[51]，本章中涉及到的缺陷类型包含 CWE78（操作系统命令注入）、CWE121（堆栈缓冲区溢出）、CWE119（内存缓冲区）、CWE-399（资源管理漏洞）。

表 5.6 实验数据集表

参数 数据集	样本数	正样本数	负样本数
CWE119	36074	8761	27313
CWE399	21062	7116	13946
CWE121	16920	4230	12690
CWE78	18673	6215	12458
Mixed	92729	26322	66407

5.2.2 实验环境

实验环境是基于 Ubuntu 18.04 操作系统，使用了 NVIDIA GeForce RTX 3080 Ti 显卡，显卡内存为 12 GB，可使用内存为 60 GB，CPU 型号为 Intel(R) Xeon(R) CPU E5-2686 v4 @ 2.30GHz，Python 版本为 3.8，CUDA 版本为 11.3，cuDNN 版本为 8，深度学习框架为 PyTorch 1.11.0，图像处理库为 torchvision 0.12.0。

5.2.3 实验结果分析

本章所设计的实验用于回答以下 3 个研究问题。

RQ1：本章方法采用基于动态切片与预训练模型的代码缺陷方法，能否适应多种的漏洞类型？

RQ2：本章方法使用 Swin Transformer 模型相对其他模型，是否具有更好的分类效果？

RQ3：本文方法相较于其他静态漏洞检测系统，是否有更好的漏洞检测性能？

针对 RQ1 的实验，分别对 CWE78、CWE121、CWE119、CWE-399 和混合数据集这五个数据集进行实验，对动态切片样本通过 CodeBERT 表示为二维张量时，张量的大小为 $L * 768$ ，其中 L 的取值为 64、128、256、512 和 1024，分别进行相应的实验。

CWE-119 数据集为内存缓冲区溢出漏洞数据集，本实验中一共提取出 36074 样本，其中正样本 8761 个，负样本 27313 个，按照 7: 2: 1 划分为训练集、验证集和测试集，这是一种常见的做法。通过这种比例设置，我们可以在训练集上进行模型的训练和调优，在验证集上选择最佳的超参数组合，并最终在测试集上对模型进行最终评估。训练过程的准确率 acc 和损失函数 loss 如图 5.2 所示。Swin Transformer 模型训练过程，学习率设置为 0.0001，迭代次数 epoch 设置为 100，每次迭代使用的样本数量 batch_size 设置为 64。通道数 in_chans 设置为 1，输出类别 num_classes 设置为 2。img_size: 输入图像的大小，是一个元组，包含两个值，分别是图像的高度和宽度。图像的高度设置为 64, 128, 256, 512, 1024，宽度为 768。window_size: Transformer 模型中每个块(block)中窗口大小设置为 8，图像的高度设置为 64、128、256、512、1024 对应的 patch_size 局部区域(patch)的大小设置为 (8, 16)、(16, 16)、(16, 16)、(32, 16)、(64, 16)。如表 5.7 所示，在高度从 64 增加到 512 时，准确率从 89.50%增加到 95.10%，增加了 5.60%。但当高度继续增加到 1024 时，准确率反而下降了，降到了 90.00%。这表明，选取合适的高度可以显著提高漏洞检测的准确率，但过大的高度并不一定会带来更好的效果，因为在当序列过短时，需要将以 0 填充在末尾，无用信

息将占据大部分输入序列的空间。高度为 512 下具有最佳的效果，因为缓冲区溢出相关的切片代码块的 token 数在 512 左右，其准确率达到了 95.1%，误报率仅有 1.6%，漏报率为 8.0%。这意味着，在高度为 512 时，可以较为准确地判断代码是否存在漏洞，同时也可以避免过多的误报情况。

表 5.7 CWE119 数据集相关漏洞实验结果

指标 长度	Acc（准确率）	FPR（误报率）	FNR（漏报率）	P（查准率）	F1- Score
64	89.5%	5.1%	16.0%	92.9%	88.3%
128	92.0%	3.8%	13.1%	95.0%	90.7%
256	93.8%	2.5%	10.3%	97.2%	93.4%
512	95.1%	1.6%	8.0%	98.6%	95.2%
1024	90.0%	4.2%	15.6%	93.1%	88.2%

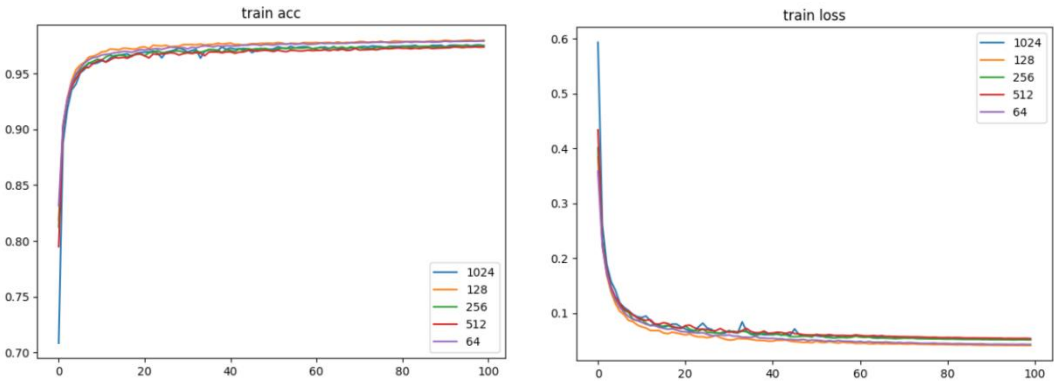


图 5.2 CWE119 训练集准确率、损失函数变化图

CWE-399 数据集为资源管理错误数据集，本实验中一共提取出 21062 样本，其中正样本 7116 个，负样本 13946 个。按照 7: 2: 1 划分为训练集、验证集和训练集，训练过程的准确率 acc 和损失函数 loss 如图 5.3 所示。随着高度的增加，准确率略有提高，误报率略有下降，漏报率稍微上升，查准率略有上升，F1 分数略有下降。这是因为高度增加可以提供更多的信息，使模型更容易准确地分类正负样本，但同时也增加了误报和漏报的可能性。在高度为 512 时，各项指标均达到最佳。准确率为 95.04%，误报率为 4.53%，漏报率为 22.61%，查准率为 98.14%，F1 分数为 0.85。这是因为长度为 512 时，模型可以获得足够的信息，同时避免了过拟合和欠拟合的情况，从而使得各项指标达到最佳。

在长度为 64 和 1024 时，准确率都较低，分别为 89.40%和 89.37%。误报率略高，漏报率稍微高一些，查准率略低，F1 分数也相应较低。这是因为长度为 64 时，CodeBERT 截断了较长的切片语句块，提供的信息不足，难以准确分类样本；而长度为 1024 时，提供的无用信息

过多。

表 5.8 CWE399 数据集相关漏洞实验结果

指标 长度	Acc（准确率）	FPR（误报率）	FNR（漏报率）	P（查准率）	F1- Score
64	89.3%	6.4%	25.7%	90.5%	81.2%
128	91.5%	4.8%	21.1%	92.9%	86.0%
256	93.4%	4.2%	16.5%	94.8%	88.2%
512	95.0%	2.3%	10.1%	97.4%	94.5%
1024	89.8%	3.2%	16.7%	95.0%	87.4%

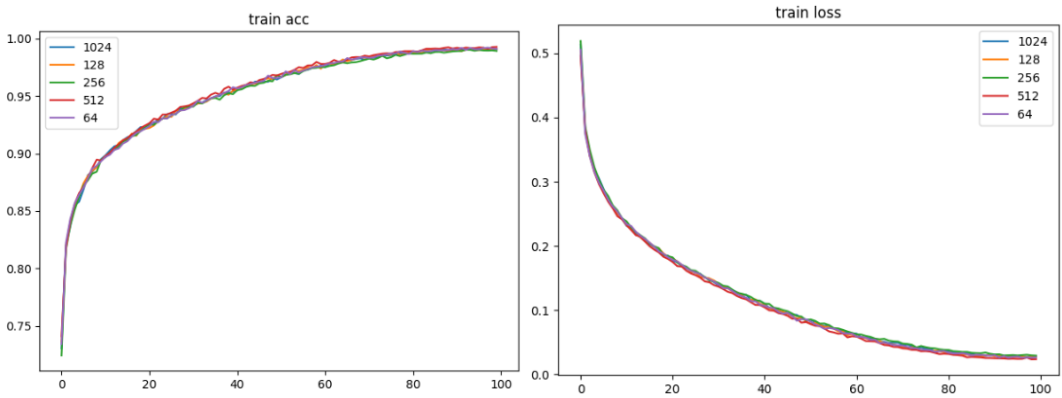


图 5.3 CWE399 训练集准确率、损失函数变化图

CWE-78 数据集为操作系统命令注入漏洞相关数据集，本实验中一共提取出 18673 样本，其中正样本 6215 个，负样本 12458 个。按照 7：2：1 划分为训练集、验证集和训练集，训练过程准确率 acc 和损失函数 loss 如图 5.4 所示。相对于高度 64 和 1024，高度 256 的准确率提高了约 5-6 个百分点，因为操作系统命令注入漏洞的切片代码块的 token 数在 256 附近，高度 256 的模型能够更好地学习到数据集的特征，准确区分正样本和负样本。另外，高度 256 时模型的查准率比较高，也就意味着模型在识别漏洞方面有较好的能力，同时能够避免误判。

表 5.9 CWE78 数据集相关漏洞实验结果

指标 长度	Acc（准确率）	FPR（误报率）	FNR（漏报率）	P（查准率）	F1- Score
64	90.1%	10.6%	7.3%	77.4%	84.5%
128	91.5%	7.2%	4.8%	83.4%	89.1%
256	96.3%	2.3%	2.7%	93.2%	95.2%
512	94.6%	1.5%	5.1%	95.6%	94.3%
1024	90.1%	3.1%	16.5%	90.7%	86.8%

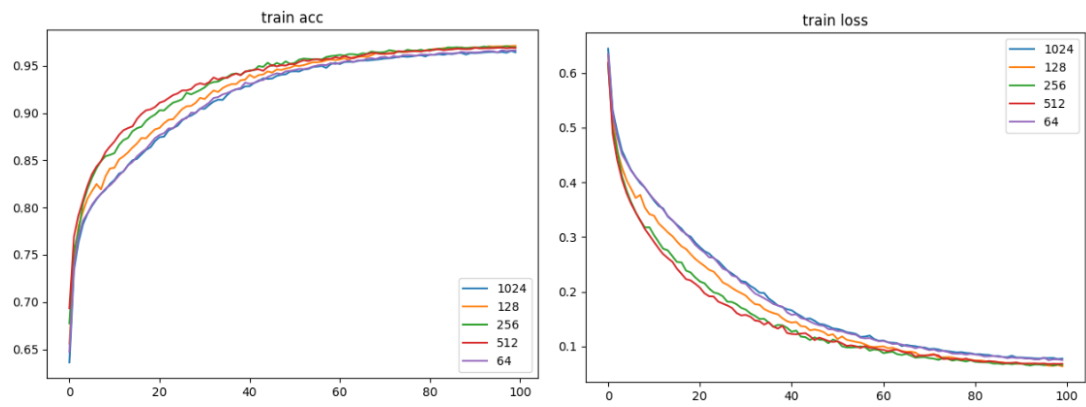


图 5.4 CWE78 训练集准确率、损失函数变化图

CWE-121 数据集为栈缓冲区溢出漏洞相关数据集，本实验中一共提取出 21062 样本，其中正样本 7116 个，负样本 13946 个。按照 7: 2: 1 划分为训练集、验证集和训练集，训练过程准确率 acc 和损失函数 loss 如图 5.5 所示。在高度为 64 和 1024 的样本中，准确率相对较低，分别在 87%左右，这是因为高度为 64 的样本包含有效信息少，难以包含足够的信息来准确检测漏洞，而长度为 1024 的样本可能太长，存在过多无关信息，导致误报率较高。在高度为 256 和 512 的样本中，准确率相对较高，分别在 96%和 94%左右，这是因为这两个长度范围内的样本包含了足够的信息来进行漏洞检测，同时也减少了误报率和漏报率。

表 5.10 CWE121 数据集相关漏洞实验结果

指标 长度	Acc（准确率）	FPR（误报率）	FNR（漏报率）	P（查准率）	F1- Score
64	87.2%	12.4%	28.4%	87.6%	79.1%
128	88.9%	10.8%	27.2%	89.2%	79.8%
256	91.3%	8.3%	22.3%	91.7%	83.7%
512	94.1%	5.6%	16.4%	94.4%	88.8%
1024	87.8%	12.2%	31.2%	88.2%	76.6%

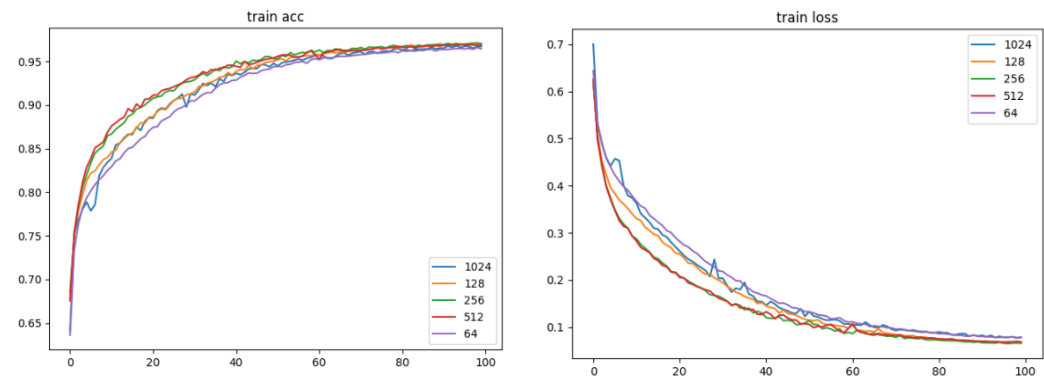


图 5.5 CWE121 训练集准确率、损失函数变化

混合数据集是将 CWE-119、CWE-399、CWE-78、CWE-121 混合的数据集，本实验中一共提取出 92729 样本，其中正样本 26322 个，负样本 66407 个。按照 7: 2: 1 划分为训练集、验证集和训练集，训练过程准确率 acc 和损失函数 loss 如图 5.6 所示。从表格可以看出，随着训练样本的增多，模型的各项指标得到了显著的提升。对于高度为 512 的样本，准确率达到 95%，误报率和漏报率均较低，F1 分数也达到了较高的值。在高度为 64 和 1024 的样本，之前由于数据样本的限制，误报率和漏报率相对较高，但是随着训练样本的增加，模型的表现也得到了一定程度的提升。

表 5.11 混合数据集相关漏洞实验结果

指标 长度	Acc（准确率）	FPR（误报率）	FNR（漏报率）	P（查准率）	F1- Score
64	91.8%	2.2%	19.1%	97.71%	89.9%
128	93.0%	1.7%	14.7%	97.31%	90.6%
256	94.2%	1.4%	11.4%	96.58%	91.3%
512	95.0%	1.1%	8.6%	96.12%	93.2%
1024	91.7%	2.3%	19.8%	97.66%	89.7%

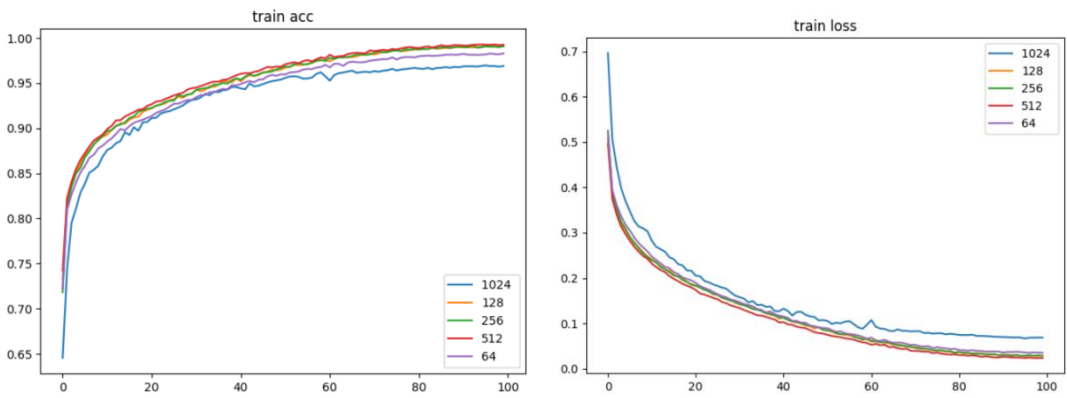


图 5.6 混合数据集准确率、损失函数变化图

针对 RQ2 的实验，对混合数据集进行实验，对动态切片样本通过 CodeBERT 表示为二维张量时，张量的大小为 $L * 768$ ，由 RQ1 的实验结果表明当 L 的取值为 512 效果较好，分别 ResNet、MobileNet、ViT、Swin Transformer 进行相应的实验。

表 5.12 不同模型在混合数据集下的实验结果

指标 模型	Acc（准确率）	FPR（误报率）	FNR（漏报率）	P（查准率）	F1- Score
ResNet	88.72%	2.78%	19.65%	91.26%	85.41%
MobileNet	91.34%	3.22%	14.58%	92.54%	88.77%
ViT	92.13%	2.85%	13.97%	92.03%	88.93%
Swin	95.04%	1.18%	8.63%	96.12%	93.2%

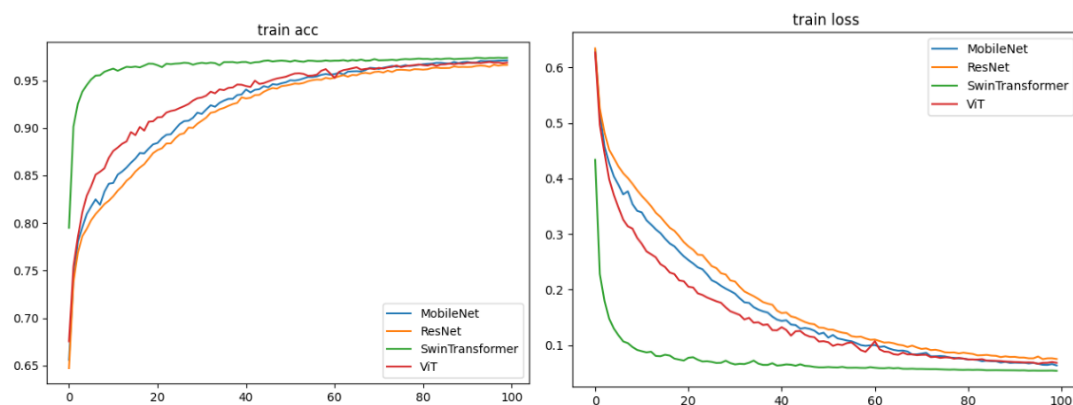


图 5.7 不同模型训练准确率、损失函数变化图

对比四个模型的结果,如表 5.12 所示,可以发现 Swin Transformer 在准确率、查准率和 F1 分数等指标上都表现优异,相对其他三个模型提升幅度较大。Swin Transformer 相对于 ResNet 提升了 6.32% 的准确率、1.86% 的查准率和 7.79% 的 F1 分数,同时降低了 1.6% 的误报率和 10.98% 的漏报率。相对于 MobileNet, Swin Transformer 提升了 3.7% 的准确率、3.58% 的查准率和 4.43% 的 F1 分数,同时降低了 2.04% 的误报率和 5.95% 的漏报率。相对于 ViT, Swin Transformer 提升了 2.91% 的准确率、4.09% 的查准率和 4.27% 的 F1 分数,同时降低了 1.67% 的误报率和 5.34% 的漏报率。

Swin Transformer 相对于其他三个模型的提升主要归因于其具有更好的特征提取能力和更强的模型表达能力。Swin Transformer 模型结构在处理大量数据上具有优势,通过类似于多层金字塔的架构对特征进行全局和局部的交互,能够更好地捕获图像中的语义信息,从而提高模型的表现能力。

针对 RQ3 的实验,在混合实验集下分别对比了 VulDeePecker、SySeVR 和 VulCNN, VulDeePecker、SySeVR 通过静态切片获取代码片段,并将其转化为序列向量表示。VulCNN 将函数的源代码转换为图像表示进而通过深度学习进行分类,通过开源代码实现上述三种方法,能够确保实验的可靠性。如表 5.13 所示,实验结果表明:本章方法,在准确率和 F1 分数有较好的表现,准确率比 VulDeePecker、SySeVR 和 VulCNN 分别高出 6.07%、3.37% 和 1.42%, F1 分数比 VulDeePecker、SySeVR 和 VulCNN 分别高出 13.38%、7.8% 和 5.02%,采用预训练模型作为基础模型可能提供了更好的语义理解能力。预训练模型通过大规模的语料库学习了丰富的语言知识,并具备了一定的上下文理解能力。这使得模型在代码漏洞检测任务中能够更好地理解代码的语义和结构,提高了检测的准确性。在查准率上略低与 VulCNN。在误报率和漏报率都明显低于其他三种方法,误报率比 VulDeePecker、SySeVR 和 VulCNN 分别低于 9.66%、4.53% 和 2.3%,漏报率比 VulDeePecker、SySeVR 和 VulCNN 分别低于 18.63%、

13.76%和 9.49%。通过捕捉程序在特定执行条件下的行为，动态切片能够更全面地检测代码中的缺陷，从而在误报率和漏报率方面取得较低的结果。

表 5.13 不同代码漏洞检测系统下各指标对比结果

Method	Acc(准确率)	FPR(误报率)	FNR(漏报率)	P(查准率)	F1- Score
VulDeePecker	88.97%	10.84%	27.26%	89.23%	79.85%
SySeVR	91.67%	5.71%	22.39%	94.29%	85.43%
VulCNN	93.62%	3.48%	18.12%	96.52%	88.21%
本章方法	95.04%	1.18%	8.63%	96.12%	93.23%

5.3 本章小结

本章主要对第三章提出的静态切片语义与度量代码的漏洞检测方法和第四章提出的基于动态切片与预训练模型的代码缺陷检测方法进行了相关实验。5.1 节中主要本章方法通过 LLVM IR 中间语言来表示语义信息和度量，与在源码表示的方法进行对比；通过加入切片度量与只有语义信息对比；通过与其他漏洞检测系统对比。5.2 节采用基于动态切片与预训练模型的代码缺陷方法，在不同的漏洞类型数据上实验；通过 Swin Transformer 模型相对其他模型对比，与其他静态漏洞检测系统对比。通过对各种评估指标的数值分析，表明了两章方法具有一定的优势。

第六章 总结与展望

6.1 工作总结

传统的漏洞检测方法通常依赖于手工设计的规则或特征，这些方法可能受限于特定的编程语言或应用程序结构，难以覆盖复杂的漏洞类型和变体。而深度学习通过学习数据中的模式和表示，具有自动提取特征和适应性学习的能力，因此被广泛应用于漏洞检测中。深度学习可以从大规模的代码库和漏洞样本中学习模式和规律，从而实现对未知漏洞的检测和预防。它能够通过对输入数据进行端到端的学习，不需要依赖领域专家手工提取特征，从而减轻了人工设计规则和特征的工作量。针对现有漏洞检测方法粒度粗和对特定语言的限制问题，本文所做的工作主要如下：

(1) 提出的基于 LLVM IR 的漏洞兴趣点检测算法。首先，针对数组变量使用、指针变量使用、算术表达式以及 API/库函数调用等四种类型的指令，总结了兴趣点选取的准则，为指令中兴趣点的选择提供了指导依据。将目标程序转化为 LLVM IR 指令的中间代码表示，并创建函数调用图记录函数间的调用关系。通过遍历函数调用图中的每个函数、函数内的基本块以及基本块内的每条 LLVM IR 指令，对指令进行六大兴趣点准则的判断。当指令满足兴趣点准则时，从切片表中获取与该指令相关的兴趣点。基于 LLVM IR 的中间表示具有平台无关性、可扩展性和中间表示优化等优势，能够更准确、全面地找到程序中的兴趣点，同时减少了对特定编程语言的依赖性。该方法能够在漏洞兴趣点检测中发挥更高的准确性和可靠性。

(2) 提出了一种基于 LLVM IR 层面的静态切片语义与度量代码的漏洞检测方法。该方法利用 SymPas 获取兴趣点的前向切片和后向切片，同时采用 Inst2vec 来获取 IR 指令的向量表示。创新地融合了指令级切片度量，包括紧密度、重叠度、覆盖度和引入了认知复杂度这一新型度量。认知复杂度度量反映了理解切片的目的数据流信息和控制流信息所需的认知努力，能够全面评估切片的质量和相关性，提高漏洞检测的准确性和可靠性。为了有效融合和学习提取的特征，采用了混合模型 ResCNN-GRU 进行训练，充分利用特征的时序和空间信息，进一步提高检测的准确性和可靠性。

(3) 提出了一种基于动态切片与预训练模型的代码漏洞检测方法。相较于仅关注程序的静态结构的静态切片，动态切片能够反映程序的实际执行情况，更真实和准确地发现只在特定执行条件下出现的缺陷。此外，动态切片方法将程序表示为更小粒度，实现了更细致的分析。并提出了基于 CodeBERT 和程序切片的代码表示算法，通过灵活调整代码表示的长度，

适应不同长度的程序切片，并确保关键信息充分保留。该算法为后续的模型训练和漏洞检测提供了方便和可靠的输入。将程序切片语句块表示为二维张量，将其视为一通道灰度图像，并将代码的结构和语义信息编码为像素值，从而全面捕捉代码的语义特征。借助计算机视觉模型 Swin Transformer 的特征提取能力，能够进一步提取图像中的关键漏洞特征，从而实现更准确的漏洞检测。

6.2 研究展望

本文分别针对静态切片和动态切片在漏洞检测领域的应用进行了研究，本文提出的方法在准确率、F1 值、精度上都有较好的表现，同时拥有更低的漏报率和误报率，能够更全面地捕捉代码的语义特征和关键漏洞特征，提高代码漏洞检测的准确性和可靠性。但考虑到诸多实际因素，本文的漏洞检测算法还存在改进的空间。在未来我们将关注以下几点：

（1）探索如何利用大模型来进行漏洞检测，利用大模型（如 ChatGPT）进行漏洞检测可以提供更强大的语言理解和表达能力，帮助发现和识别代码中的漏洞模式。首先使用大模型在大规模通用数据上进行预训练，然后通过漏洞数据集上进行微调，使其适应特定的漏洞检测任务。这样可以利用大模型的语言理解和表达能力来学习漏洞的模式和特征。然而，需要充分考虑数据集构建、模型训练、特征表示、模型评估与优化等关键问题，以实现高效、准确的漏洞检测系统；

（2）针对多种编程语言，可以设计通用的代码表示方法。通过对不同语言的代码进行转化和标准化，将其转化为通用的表示形式，以便深度学习模型能够在不同语言之间进行知识迁移和共享。这样可以利用大规模的跨语言代码数据进行训练，提高模型的泛化能力和鲁棒性。

（3）对代码的多模态表示，可以将代码与其他类型的信息结合起来，例如文本描述、注释、执行轨迹等。通过融合多种信息来源，可以提供更全面、多维度的代码表示，增强深度学习模型对漏洞的检测能力。

参考文献

- [1] 李珍,邹德清,王泽丽,金海.面向源代码的软件漏洞静态检测综述[J].网络与信息安全学报,2019,5(01):1-14.
- [2] Baldoni R, Coppa E, D'elia D C, et al. A survey of symbolic execution techniques[J]. ACM Computing Surveys (CSUR), 2018, 51(3): 1-39.
- [3] Grigorenko E L, Sternberg R J. Dynamic testing[J]. Psychological bulletin, 1998, 124(1): 75.
- [4] Pereira J D A, Vieira M. On the use of open-source C/C++ static analysis tools in large projects[C].2020 16th European Dependable Computing Conference (EDCC). IEEE, 2020: 97-102.
- [5] Aniqua Z B, Tamara D. IDE plugins for detecting in-put-validation vulnerabilities[C].IEEE Symposium on Security and Privacy Workshops. 2017: 143-146.
- [6] Zhaowei Z S S. Detection of array bound overflow by interval set based on cppcheck[J]. Journal of Computer Applications, 2013, 33(11): 3257.
- [7] Walden J, Doyle M. Savi: Static-analysis vulnerability indicator[J]. IEEE Security & Privacy, 2012, 10(3): 32-39.
- [8] Baloglu B. How to find and fix software vulnerabilities with coverity static analysis[C].2016 IEEE Cybersecurity Development (SecDev). IEEE, 2016: 153-153.
- [9] Shahriar H, Zulkernine M. Mitigating program security vulnerabilities: Approaches and challenges[J]. ACM Computing Surveys (CSUR), 2012, 44(3): 1-46.
- [10] Ramos D A, Engler D. Under-constrained symbolic execution: Correctness checking for real code[C].24th {USENIX} Security Symposium ({USENIX} Security 15). 2015: 49-64.
- [11] Liang H, Wang L, Wu D, et al. MLSA: a static bugs analysis tool based on LLVM IR[C].2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD). IEEE, 2016: 407-412.
- [12] Ganesh V, Leek T, Rinard M. Taint-based directed whitebox fuzzing[C].2009 IEEE 31st International Conference on Software Engineering. IEEE, 2009: 474-484.
- [13] Zhang Y, Huo W, Jian K, et al. SRFuzzer: an automatic fuzzing framework for physical SOHO router devices to discover multi-type vulnerabilities[C].Proceedings of the 35th annual computer security applications conference. 2019: 544-556.
- [14] Amin A, Eldessouki A, Magdy M T, et al. Androshield: Automated android applications vulnerability detection, a hybrid static and dynamic analysis approach[J]. Information, 2019, 10(10): 326.
- [15] Padmanabhuni B M, Tan H B K. Auditing buffer overflow vulnerabilities using hybrid static - dynamic analysis[J]. IET Software, 2016, 10(2): 54-61.
- [16] Nguyen V H, Tran L M S. Predicting vulnerable software components with dependency graphs[C].Proceedings of the 6th International Workshop on Security Measurements and Metrics. 2010: 1-8.
- [17] Shin Y, Meneely A, Williams L, et al. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities[J]. IEEE transactions on software engineering, 2010, 37(6): 772-787.
- [18] Perl H, Dechand S, Smith M, et al. Vccfinder: Finding potential vulnerabilities in open-source projects to assist code audits[C].Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. 2015: 426-437.
- [19] Hin D, Kan A, Chen H, et al. LineVD: statement-level vulnerability detection using graph neural networks[C].Proceedings of the 19th International Conference on Mining Software Repositories. 2022: 596-607.
- [20] Yamaguchi F, Lottmann M, Rieck K. Generalized vulnerability extrapolation using abstract syntax

- trees[C].Proceedings of the 28th annual computer security applications conference. 2012: 359-368.
- [21]Li Z, Zou D, Xu S,et al.VulDeePecker:A Deep Learning-Based System for Vulnerability Detection [C].Network and Distributed System Security Symposium.2018.ndss.2018.23165.
- [22]Li Z, Zou D, Xu S, et al. Sysevr: A framework for using deep learning to detect software vulnerabilities[J]. IEEE Transactions on Dependable and Secure Computing, 2021, 19(4): 2244-2258.
- [23]Yamaguchi F, Golde N, Arp D, et al. Modeling and discovering vulnerabilities with code property graphs[C].2014 IEEE Symposium on Security and Privacy. IEEE, 2014: 590-604.
- [24]Zhou Y, Liu S, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in neural information processing systems, 2019, 32.
- [25]Cao S, Sun X, Bo L, et al. MVD: memory-related vulnerability detection based on flow-sensitive graph neural networks[C].Proceedings of the 44th International Conference on Software Engineering. 2022: 1456-1468.
- [26]Wu Y, Zou D, Dou S, et al. VulCNN:an image-inspired scalable vulnerability detection system[C].P roceedings of the 44th International Conference on Software Engineering. 2022: 2365-2376.
- [27]Rabheru R, Hanif H, Maffeis S. A hybrid graph neural network approach for detecting PHP vulnera bilities[C].2022 IEEE Conference on Dependable and Secure Computing (DSC). IEEE, 2022: 1-9.
- [28]Lin G, Wen S, Han Q L, et al. Software vulnerability detection using deep neural networks: a surv ey[J].Proceedings of the IEEE, 2020, 108(10): 1825-1848.
- [29]Gu J, Wang Z, Kuen J, et al. Recent advances in convolutional neural networks[J]. Pattern recogniti on, 2018, 77: 354-377.
- [30]Rao R M, Liu J, Verkuil R, et al. MSA transformer[C].International Conference on Machine Learni ng. PMLR, 2021: 8844-8856.
- [31]Okada S, Ohzeki M, Taguchi S. Efficient partition of integer optimization problems with one-hot en coding[J]. Scientific reports, 2019, 9(1): 13036.
- [32]Zhao R, Mao K. Fuzzy bag-of-words model for document representation[J]. IEEE transactions on fu zzy systems, 2017, 26(2): 794-804.
- [33]Qaiser S, Ali R. Text mining: use of TF-IDF to examine the relevance of words to documents[J]. I nternational Journal of Computer Applications, 2018, 181(1): 25-29.
- [34]陈振强. 基于依赖性分析的程序切片技术研究 [D]; 东南大学, 2003.
- [35]李必信. 程序切片技术及其应用 [M]. 程序切片技术及其应用, 2006.
- [36]Weiser M. Program slicing [J]. IEEE Transactions on software engineering, 1984, (4): 352-7.
- [37]Ottensstein K J, Ottensstein L M. The program dependence graph in a software development environ ment [J]. ACM Sigplan Notices, 1984, 19(5): 177-84.
- [38]Horwitz S, Reps T, Binkley D. Interprocedural slicing using dependence graphs [J]. ACM Transacti ons on Programming Languages and Systems (TOPLAS), 1990, 12(1): 26-60.
- [39]Zhang Y-Z. SymPas: symbolic program slicing [J]. Journal of Computer Science and Technology, 2 021, 36: 397-418.
- [40]Fosdick L D, Osterweil L J. Data flow analysis in software reliability[J]. ACM Computing Surveys (CSUR), 1976, 8(3): 305-330.
- [41]Lattner C. LLVM and Clang: Next generation compiler technology[C].The BSD conference. 2008, 5: 1-20.
- [42]Ruwase O, Lam M S. A Practical Dynamic Buffer Overflow Detector[C].NDSS. 2004, 2004: 159-16 9.
- [43]Hovemeyer D, Pugh W. Finding more null pointer bugs, but not too many[C].Proceedings of the 7t h ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools and engineering. 200 7: 9-14.
- [44]Halfond W G, Viegas J, Orso A. A classification of SQL-injection attacks and countermeasures[C].P

- roceedings of the IEEE international symposium on secure software engineering. IEEE, 2006, 1: 13-15.
- [45]Schmidt A D, Clausen J H, Camtepe A, et al. Detecting symbian os malware through static function call analysis[C].2009 4th International Conference on Malicious and Unwanted Software (MALWARE). IEEE, 2009: 15-22.
- [46]Cummins C, Fisches Z V, Ben-Nun T, et al. Programl: A graph-based program representation for data flow analysis and compiler optimizations[C].International Conference on Machine Learning. PMLR, 2021: 2244-2253.
- [47]闫丽. 基于切片度量的软件模块复杂性研究[D]. 南京邮电大学, 2017.
- [48]Chung J, Gulcehre C, Cho K, et al. Gated feedback recurrent neural networks[C].International conference on machine learning. PMLR, 2015: 2067-2075.
- [49]He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C].Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
- [50]Zhu Z, Dai W, Hu Y, et al. Speech emotion recognition model based on Bi-GRU and Focal Loss [J]. Pattern Recognition Letters, 2020, 140: 358-365.
- [51]Software Assurance Reference Dataset.[EB/OL].(2023-03-20).<https://samate.nist.gov/SRD/index.php>.
- [52]Agrawal H, Horgan J R. Dynamic program slicing[J]. ACM SIGPlan Notices, 1990, 25(6): 246-256.
- [53]Cadarc C, Dunbar D, Engler D R. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs[C].OSDI. 2008, 8: 209-224.
- [54]Kanade A, Maniatis P, Balakrishnan G, et al. Learning and evaluating contextual embedding of source code[C].International conference on machine learning. PMLR, 2020: 5110-5121.
- [55]Liu Z, Lin Y, Cao Y, et al. Swin transformer: Hierarchical vision transformer using shifted windows[C].Proceedings of the IEEE/CVF international conference on computer vision. 2021: 10012-10022.